Руководство автора Primula

Primula Documenting System 0.3.0.dev9 · редакция 28.10.2026

Содержание

Руководство автора Primula....6
Страницы....6
Что нужно....6
Как устроен документ....7
1. Первый документ....8
1.1. Файл....8
1.2. Заголовок YAML....8
1.3. Текст и абзацы....8
1.4. Заголовки, пункты, списки....9
1.5. Проверка и сборка....9
1.6. Оформление по умолчанию....9
2. Договор....10
2.1. Целиком....10
2.2. Данные — в заголовке, не в тексте....11
2.3. Начало договора....11
2.4. Разделы и пункты....11
2.5. Ссылки на пункты....11
2.6. Суммы и числа....12
2.7. Реквизиты и подписи....12
2.8. Приложение....12
2.9. Сноски....12
2.10. Повторяющиеся блоки....12
3. Акт и таблицы с итогами....13
3.1. Простая таблица....13
3.2. Акт: строки из данных и итог....13
3.3. Таблица без цикла....14
3.4. Ячейка из нескольких абзацев....14
3.5. Оформление таблицы....14
4. Служебное письмо....16
4.1. Пример....16
4.2. Части письма....16
4.3. Свои отличия....17
5. Техническая документация....18
5.1. Предупреждения и примечания....18
5.2. Варианты для разных исполнений....18
5.3. Сокращения....19
5.4. Указатель, оглавление, перечни....19
5.5. Рисунки....19
5.6. Текст программ и интерфейс....20

5.7. Общие фрагменты и ключи....20
6. Проект из нескольких документов....21
6.1. Файл проекта....21
6.2. Документ проекта....21
6.3. Фрагменты: общий текст....22
6.4. Модули: макросы и функции....22
6.5. Данные из файлов....23
6.6. Сборка проекта....23
6.7. Экспорт в Markdown, HTML, DOCX....24
6.8. Сводный документ....24
6.9. Ссылки на другой документ....24
6.10. Карта ключей....24
6.11. Проект заказа....24
7. Стили....25
7.1. Выбор набора....25
7.2. Применение стилей в тексте....25
7.3. Свой модуль стилей....26
7.4. Слои....26
7.5. Страницы и колонтитулы....27
7.6. Переход с ODT Markdown Kit....27
8. Сообщения транслятора....28
8.1. Как читать сообщение....28
8.2. Предупреждения и выпуск....28
8.3. Частые ошибки....28
8.4. Если непонятно, что не так....30
9. Параллельные тексты....31
9.1. Пример....31
9.2. Потоки....31
9.3. Сегменты....32
9.4. Нумерация, ссылки, сноски....32
9.5. Проверка соответствия....33
9.6. Перевод: заготовки, подтверждение, актуальность....33
9.7. Протокол разногласий....34
9.8. Стили....34
10. Код: диалект Python....35
10.1. Где пишется код....35
10.2. Числа: decimal вместо int и float....35
10.3. Строки....35
10.4. Списки, кортежи, словари....36
10.5. Записи....36
10.6. Даты и длины....37

10.7. Функции..... 37
10.8. Глобальные имена: ленивые и неизменяемые..... 38
10.9. Модули и фрагменты..... 38
10.10. Значения в тексте и макросы..... 39
10.11. Чего в ядре нет..... 40
11. Библиотеки: разработка и подключение..... 41
11.1. Что может библиотека..... 41
11.2. Устройство каталога..... 41
11.3. Модуль..... 42
11.4. Тесты..... 42
11.5. Подключение к проекту..... 43
11.6. Данные с датой действия..... 43
11.7. Выпуск..... 44
Приложение А. Конструкции..... 45
Команды..... 45
Краткие формы..... 47
Маркеры закрытия..... 48
Приложение Б. Функции выражений..... 49
Числа..... 49
Деньги и НДС..... 49
Прописью..... 50
Даты и время..... 50
Единицы и длины..... 51
Текст..... 51
Списки и записи..... 51
Данные..... 52
Математика (std/math)..... 52
QR-коды и кодировки..... 53
Проверки и документ..... 53
Формулы таблиц..... 53
Код стилей..... 54
Проект..... 55
Методы..... 55
Приложение В. Ключи YAML-заголовка..... 57
Приложение Г. Сообщения транслятора..... 60
Приложение Д. Стили комплекта..... 71
std/styles/default..... 71
std/styles/business..... 73
std/styles/letter..... 74
std/styles/technical..... 75
std/styles/report..... 75

std/styles/compact....76
Глоссарий терминов Primula (ru / uk / en)....78

Руководство автора Primula

Руководство для тех, кто пишет документы на Primula: договоры, акты, письма, инструкции и отчёты. Оно объясняет, как решить типичную задачу, и не повторяет справочные таблицы. Полный перечень конструкций, функций, ключей заголовка, сообщений и стилей дан в приложениях, которые генерируются из транслятора и всегда соответствуют его версии.

Нормативный документ — спецификация `primula-spec-v1.1-draft.ru.md` и описание стандартной библиотеки `primula-stdlib-v1.1-draft.ru.md` (переводы на украинский и английский лежат рядом: `….uk.md`, `….en.md`). В начале каждой страницы руководства в комментарии указаны разделы спецификации, на которые она опирается.

Страницы

Страница	О чём
Первый документ	файл `.prml`, заголовок YAML, абзацы, заголовки, пункты, проверка и сборка
Договор	параметры, место и дата, преамбула, разделы и пункты, ссылки на пункты, суммы прописью, реквизиты, приложение
Акт и таблицы с итогами	таблицы, столбцы с типами, формулы и итоги, строки из данных
Служебное письмо	набор стилей `letter`: бланк, адресат, номер и дата, подпись
Техническая документация	предупреждения, условные варианты, сокращения, указатель, рисунки
Проект из нескольких документов	`project.p.prml`, общие параметры, фрагменты, модули, цели сборки, заказы
Стили	наборы стилей комплекта, свой модуль стилей, назначение стилей, переход с Kit
Сообщения транслятора	как читать сообщения, частые ошибки и что делать
Параллельные тексты	двуязычный договор, протокол разногласий: потоки и сегменты, язык и нумерация потока, проверка соответствия, подтверждение перевода
Код: диалект Python	числа `decimal`, строки, коллекции, записи, даты, функции, ленивые неизменяемые глобальные имена, модули и экспорт из фрагмента (`@export`), значения в тексте, чего в ядре нет
Библиотеки: разработка и подключение	библиотека из модулей и данных, что ей доступно и чего нет, устройство каталога, документы-тесты с `fail()`, подключение ссылкой, копией или подмодулем Git (решение 136), данные с датой действия, выпуск

Приложения (генерируются, вручную не правятся): конструкции, функции выражений, ключи заголовка, сообщения, стили комплекта. Термины — по глоссарию.

Что нужно

- Транслятор `primula` — из пакета или из дерева исходников (`./primula`). Для PDF нужен LibreOffice.
- Редактор: Primula Studio (подсветка, подсказки, проверка, просмотр PDF) или любой текстовый редактор с UTF-8.

Studio описана в своей справке (F1 в Studio); руководство показывает команды транслятора, которые Studio выполняет за вас.

Как устроен документ

Документ — обычный текстовый файл. Текст пишется как есть, а всё, что относится к разметке, начинается с вертикальной черты `|`: `|<` — заголовок, `|>` — пункт договора, `|(сума)` — значение из кода, `|*{важно}` — выделение. Транслятор проверяет файл, вычисляет значения, нумерует пункты и ссылки и записывает документ ODT (а через LibreOffice — PDF). Оформление задаётся стилями, а не в тексте: тот же документ печатается иначе, если подключить другой набор стилей.

1. Первый документ

1.1. Файл

Документ Primula — текстовый файл в кодировке UTF-8 с расширением `.prml`. Рядом с документом могут лежать файлы других видов; их различает подрасширение:

Файл	Вид
`договір.prml`	документ — из него собирается ODT
`умови.f.prml`	фрагмент — часть текста для вставки `|include`
`бізнес.m.prml`	модуль — макросы и функции для `|import`
`стилі.s.prml`	модуль стилей
`project.p.prml`	файл проекта

1.2. Заголовок YAML

Файл начинается с заголовка между строками `---`. В нём — данные документа, без вычислений:

```
---
doc:
  title: "Довідка про співпрацю"
  language: uk-UA
params:
  організація: "ТОВ «Приклад»"
  рік: 2026
---
|(організація) співпрацює з нами з |(рік) року.
```

- `doc:` — метаданные ODT: название, язык, автор, номер, редакция и т. д. В тексте они доступны как `|(doc.title)`. Язык `uk-UA` определяет кавычки, переносы, формат дат и сумм прописью.
- `params:` — параметры документа. В тексте и коде они доступны по имени. При сборке параметр можно заменить: `primula build довідка.prml --param рік=2027`.

Полный перечень ключей — в приложении «Ключи заголовка».

1.3. Текст и абзацы

Абзацы разделяются пустой строкой. Перевод строки внутри абзаца — пробел, поэтому длинный абзац можно переносить как удобно.

Всё служебное начинается с `|`. Что идёт за чертой, определяет конструкцию:

- буква — команда: `|toc`, `|table(…)`;
- `(` — значение выражения: `|(організація)`;
- знак препинания — короткая форма: `|<` заголовок, `|>` пункт, `|*{…}` полужирный;
- вторая черта — сама черта: `||`.

Выделения внутри строки (заполнитель в этом примере даёт ожидаемое предупреждение P2635):

```
Строк дії — |*{до 31.12.2026}, умови — |/{див. додаток}, код |`{UA-17}.
```

```
Назва документа в лапках мови документа: |"{Положення про закупівлі}.
Незаповнене місце в чернетці: |?{прізвище}.
```

Кавычки `|"{…}` ставятся по языку документа («…» для украинского). Заполнитель `|?{…}` подсвечивается и даёт предупреждение при проверке: так черновик не уйдёт в выпуск незаметно (`--strict` превращает предупреждение в ошибку).

1.4. Заголовки, пункты, списки

```
|<(number=False) Положення про відрядження
|< Загальні положення
|> Положення визначає порядок направлення працівників у відрядження.
|> Строк відрядження не перевищує 30 календарних днів.
|>> Строк може бути продовжено наказом керівника.
|< Документи
Працівник отримує:
|- посвідчення про відрядження;
|- кошторис витрат.
```

- `|<` — заголовок первого уровня, `|<<` — второго, и так до шести. Номера ставит транслятор; `number=False` — заголовок без номера (название документа).
- `|>` — пункт, `|>>` — подпункт. Пункты нумеруются внутри раздела: «1.1.», «1.2.», «1.2.1.» (вид номеров задаёт набор стилей: в отчёте по ДСТУ 3008 — без точки в конце).
- `|-` — маркированный элемент списка, `|#` — нумерованный; `|--`, `|##` — следующий уровень.

Пункт, заголовок и элемент списка — строчные формы: они заканчиваются в конце строки. Если пункт состоит из нескольких абзацев, его пишут окружением и закрывают `|end>`:

```
|>:
Перший абзац пункту.

Другий абзац того ж пункту.
|end>
```

Список и таблица сразу после пункта входят в него; обычный абзац после пункта — уже не часть пункта.

1.5. Проверка и сборка

```
primula check довідка.prml          # проверка без записи файла
primula build довідка.prml          # довідка.odt рядом с исходником
primula build довідка.prml --pdf    # ещё и PDF (нужен LibreOffice)
```

Проверка сообщает об ошибках и предупреждениях с указанием строки и столбца; что делать с ними — на странице «Сообщения транслятора». В Studio те же команды — «Проверить документ» (`Ctrl+Shift+Enter`) и «Собрать документ» (`F9`); сообщение открывает нужную строку щелчком.

1.6. Оформление по умолчанию

Если документ не называет стили, применяется встроенный набор `std/styles/default`: Liberation Sans 11 pt, поля A4, оформление заголовков, таблиц, сносок. Для деловых документов, писем, инструкций и отчётов в комплекте есть готовые наборы; их подключает одна строка заголовка, например `styles: [std/styles/business]`. Подробнее — на странице «Стили».

2. Договор

Страница показывает, как собрать договор: данные в заголовке, текст с нумерованными пунктами, ссылки на пункты вместо набранных вручную номеров, суммы прописью, реквизиты сторон и приложение. Оформление — набор стилей `std/styles/business`.

2.1. Целиком

```
---
doc:
  title: "Договір про надання послуг"
  identifier: "П-2026/017"
  language: uk-UA
styles: [std/styles/business]
params:
  місто: "м. Харків"
  дата: 2026-10-12
  строк_днів: 5
  виконавець: {назва: "ТОВ «Виконавець»", код: "00000000", особа: "директора Петренка
Петра Петровича", підписант: "П. П. Петренко"}
  замовник: {назва: "ТОВ «Замовник»", код: "11111111", особа: "директора Іваненка
Івана Івановича", підписант: "І. І. Іваненко"}
---
|code:
послуги = [
    record(назва="Налаштування обладнання", кількість=dec("2"), ціна=dec("1500.00")),
    record(назва="Навчання персоналу", кількість=dec("1"), ціна=dec("2400.00")),
]
сума = sum([п.кількість * п.ціна for п in послуги])
|end
|.DocumentTitle: ДОГОВІР № |(doc.identifier)
|.DocumentSubtitle: про надання послуг
|.DatePlace: |(місто), |(fmt_date(дата, "contract"))

|.Preamble: |(виконавець.назва) (далі — |.DefinedTerm{Виконавець}) в особі |
(виконавець.особа) та |(замовник.назва) (далі — |.DefinedTerm{Замовник}) в особі |
(замовник.особа) уклали цей договір про таке.

|<(id="предмет") Предмет договору
|>(id="послуги") Виконавець надає послуги, перелічені в |@("специфікація", "label"), а
Замовник приймає й оплачує їх.

|<(id="ціна") Ціна і розрахунки
|> Ціна послуг становить |(money_full(сума, "UAH")).
|>(id="оплата") Замовник оплачує послуги протягом |(строк_днів) |(plural(строк_днів,
"банківського дня", "банківських днів", "банківських днів"))|footnote{Банківський день
— день, у який банки України здійснюють розрахунки.} з дати підписання акта.

|<(id="відповідальність") Відповідальність сторін
|> За прострочення оплати (|@("оплата", "label")) Замовник сплачує пеню в розмірі
подвійної облікової ставки НБУ.

|<(number=False) Реквізити і підписи сторін
|table(style="TableParties", header=1):
|: Виконавець |& Замовник
|: |(виконавець.назва) |& |(замовник.назва)
|: Код ЄДРПОУ |(виконавець.код) |& Код ЄДРПОУ |(замовник.код)
|: _____ |(виконавець.підписант) |& _____ |(замовник.підписант)
|end table

|appendix(id="специфікація", title="Специфікація послуг", subtitle=f"до Договору №
{doc.identifier}"):
|table(style="TableServices", header=1, footer=1, cols=["52%", col("12%", id="к",
type="decimal", align="right"), col("18%", id="ц", type="money", align="right"),
```

```
col("18%", id="c", type="money", currency="UAH", align="right")]):
|: Послуга |& Кількість |& Ціна, грн |& Сума, грн
|for(п in послуги):
|: |(п.назва) |& |(п.кількість) |& |(п.ціна) |& |=(this_row("к") * this_row("ц"))
|endfor
|:|&(span=3, align="right") Разом |& |=(sum(column("c", rows=body_rows())))
|end table
|end appendix
```

Дальше — по частям.

2.2. Данные — в заголовке, не в тексте

Всё, что меняется от договора к договору, записывается в `params:` заголовка: стороны, город, дата, сроки. В тексте значения выводятся выражением `|(…)`: `|(місто)`, `|(виконавець.назва)`. Вложенная запись (`виконавець: {назва: …, код: …}`) читается через точку.

Так текст не нужно править при смене стороны, а Studio показывает параметры отдельной формой. Если договоров много, общие данные выносят в проект (страница «Проект»).

Не вписывайте в шаблон настоящие коды, IBAN и подписи — пока данных нет, используйте заполнитель `|?{…}`: проверка напомнит о нём, а `--strict` не даст собрать выпуск.

2.3. Начало договора

Название, подзаголовок, место и дата, преамбула — абзацы со стилями набора `business`: `|.DocumentTitle: …`, `|.DatePlace: …`, `|.Preamble: …`. Стиль, записанный в начале строки через `|.Имя:`, относится ко всему абзацу этой строки; `|.Имя{…}` — к фрагменту внутри строки (здесь `|.DefinedTerm{Виконавець}` выделяет определяемый термин).

Дата «прописью договора» — функция `fmt_date(дата, "contract")`: «12» жовтня 2026 р. Другие форматы дат, а также `fmt_date(дата, "long")` и `"short"` — в приложении «Функции выражений».

2.4. Разделы и пункты

`|<` — раздел, `|>` — пункт, `|>>` — подпункт. Номера ставит транслятор, поэтому вставка нового пункта в середину не требует ручной перенумерации.

2.5. Ссылки на пункты

Никогда не пишите номер пункта вручную. Пункту или разделу, на который ссылаются, дают `id=`, а в тексте пишут ссылку:

Запись	Текст в документе
`|@("оплата")`	2.2
`|@("оплата", "label")`	п. 2.2
`|@("ціна", "label")`	розд. 2
`|@("специфікація", "label")`	дод. 1
`|@("ціна", "title")`	Ціна і розрахунки

Ссылка может стоять раньше цели. Если пункт удалён или его `id=` изменён, проверка сообщит о неизвестной цели — висящих ссылок в документе не бывает. Падежная форма метки — `|@("ціна", "label", case="loc")` → «розділі 2».

2.6. Суммы и числа

Деньги в Primula — десятичные числа: пишите `dec("1500.00")` или параметр заголовка, а не дробь с плавающей точкой. Вывод — функциями библиотеки:

```
|code:
сума = dec("5400.00")
|end
|(money(сума, "UAH")) — |(money_full(сума, "UAH")).
ПДВ 20 %: |(money(vat(сума, 20), "UAH")), разом з ПДВ |(money(with_vat(сума, 20),
"UAH")).
Строк — |(words(5)) |(plural(5, "день", "дні", "днів")).
```

`money` даёт «5 400,00 грн», `money_full` — сумму цифрами и прописью, `words` — число словами, `plural` — форму слова для числа. Число или дата, выведенные без функции форматирования, дают предупреждение: так в договор не попадёт «5400.0».

2.7. Реквизиты и подписи

Реквизиты сторон — таблица без линий `TableParties` (две колонки по 50 %): первая строка — названия сторон, дальше — по строке на реквизит. Строка подписи — в той же таблице, поэтому подписи сторон стоят на одном уровне.

2.8. Приложение

`|appendix(…):` … `|end appendix` начинает приложение с новой страницы, выводит «Додаток 1», подзаголовок и название и открывает свою нумерацию разделов, рисунков и таблиц. Ссылка `|@("специфікація", "label")` даёт «дод. 1». Таблица с итогом в приложении разобрана на странице «Акт и таблицы с итогами».

2.9. Сноски

`|footnote{текст}` ставит сноску внизу страницы; номер считает Writer.

2.10. Повторяющиеся блоки

Если блок реквизитов или подписи повторяется во многих договорах, его оформляют макросом в модуле (`|def`, страница «Проект») или фрагментом (`|include`). Пример из комплекта — `examples/02-contract` (модуль `lib/business.m.prml` с макросами реквизитов и подписи).

3. Акт и таблицы с итогами

Таблица — окружение `|table(…):` … `|end table`. Строка начинается с `|:`, ячейки разделяет `|&`. Транслятор считает итоги по формулам, проверяет число ячеек в строках и оформляет числа и суммы по языку документа.

3.1. Простая таблица

```
|table(cols=["40%", "*"], header=1):
|: Показник |& Значення
|: Строк поставки |& 10 робочих днів
|: Місце поставки |& склад Покупця
|end table
```

- `header=1` — одна строка заголовка; на следующих страницах она повторяется.
- `cols` — ширины столбцов: `"40mm"` — фиксированная, `"30%"` — доля таблицы, `"*"` — остаток, `"auto"` — по содержимому, `"20mm..60mm"` — по содержимому в пределах.
- Число ячеек каждой строки должно совпадать с числом столбцов; лишняя или недостающая ячейка — ошибка с номером строки. Пустая ячейка — `|&` без текста.

3.2. Акт: строки из данных и итог

Пусть позиции акта лежат в CSV:

```
назва,кількість,ціна
Налаштування обладнання,2,1500.00
Навчання персоналу,1,2400.00
Виїзд спеціаліста,3,350.00
```

Таблица акта с итогом и НДС:

```
|code:
позиції = load_csv("data/позиції.csv", types={"кількість": "decimal", "ціна":
"money"})
|end
|table(id="т-акт", caption="Перелік наданих послуг", header=1, footer=3, cols=["10mm",
"*", col("18mm", id="к", type="decimal", align="right"), col("26mm", id="ц",
type="money", align="right"), col("30mm", id="с", type="money", currency="UAH",
align="right")]):
|: № |& Послуга |& К-сть |& Ціна |& Сума
|for(н, п in enumerate(позиції, 1)):
|: |(н) |& |(п.назва) |& |(п.кількість) |& |(п.ціна) |& |=(this_row("к") *
this_row("ц"))
|endfor
|:|&(span=4, align="right") Разом без ПДВ |& |=(sum(column("с", rows=body_rows())))
|:|&(span=4, align="right") ПДВ 20 % |& |=(round(cell(len(позиції) + 2, "с") *
dec("0.20"), 2))
|:|&(span=4, align="right") |*{Усього} |& |=(cell(len(позиції) + 2, "с") +
cell(len(позиції) + 3, "с"))
|end table
Див. |@("т-акт", "label").
```

Что здесь происходит:

- `load_csv` читает файл; `types=` объявляет типы столбцов, иначе всё читается строками. Путь — относительно документа и не выходит за корень проекта. Так же работают `load_yaml` и `load_json`.

- `col(ширина, id=…, type=…, currency=…, align=…)` описывает столбец. `id` — имя столбца в формулах. `type="money"` выводит значение с двумя знаками («1 500,00»), `currency="UAH"` добавляет «грн», `type="decimal"` — число с разделителями групп.
- `|for(…):` … `|endfor` повторяет строку для каждой позиции.
- `|=(…)` — формула ячейки. Она вычисляется, когда известны все обычные ячейки: `this_row("к")` — значение столбца `к` этой строки, `column("с", rows=body_rows())` — столбец по строкам тела (без заголовка и итогов), `cell(строка, столбец)` — ячейка по номеру строки (с 1, считая заголовок) и столбцу.
- `footer=3` — три строки итогов; они не входят в `body_rows()`.
- `|:|&(span=4, align="right")` — аргументы первой ячейки пишутся вплотную после `|:`; `span=4` объединяет четыре столбца.
- `caption=` и `id=` дают подпись «Таблиця 1 — Перелік наданих послуг» и цель ссылки «табл. 1».

Деньги считаются десятичными числами, поэтому копейки не теряются. Округление — `round(x, 2)`; правила округления бухгалтерских сумм и НДС — функции `round_money`, `vat`, `with_vat` библиотеки (приложение «Функции»).

3.3. Таблица без цикла

Если строки — это записи целиком, цикл не нужен:

```
|code:
склад = [record(назва="Кабель", к=dec("120")), record(назва="Роз'єм", к=dec("40"))]
|end
|table(data=склад, fields=["назва", "к"], titles=["Найменування", "Кількість"],
cols=["*", col("30mm", id="к", type="decimal", align="right")], footer=1):
|:|&(align="right") Разом |& |=(sum(column("к", rows=body_rows())))
|end table
```

3.4. Ячейка из нескольких абзацев

Компактная строка `|:` умещается в одной строке исходника. Для ячейки со списком или несколькими абзацами — развёрнутая форма:

```
|table(cols=["35%", "*"], header=0):
|row:
|cell: Обов'язки Виконавця
|cell:
Виконавець зобов'язується:

|- надати послуги в строк;
|- передати звіт.
|end cell
|end row
|end table
```

Компактные и развёрнутые строки можно смешивать в одной таблице.

3.5. Оформление таблицы

Вид таблицы задаёт её стиль: `|table(style="TableCompact", …)`. В наборах стилей комплекта есть готовые: `TableGrid` (по умолчанию), `TableCompact`, `TableNoBorders`; в

`business` — `TableServices`, `TableSchedule`, `TableForm`, `TableParties` с заданными ширинами столбцов (свои `cols` таблицы их заменяют). Разовые отличия — аргументы таблицы, строки (`|:(background="#EEEEEE")`) и ячейки (`|&(align="center")`): стиль таблицы → столбец → строка → ячейка, каждый уровень уточняет предыдущий. Линии — `frame=` (рамка) и `rules=` (`"all"`, `"rows"`, `"cols"`, `"none"`).

Таблица внутри раздела с колонками (`|section(cols=2)`) получает ширину колонки.

4. Служебное письмо

Письмо оформляется набором стилей `std/styles/letter`. Он построен на деловом наборе `business` и следует ДСТУ 4163:2020: поля — левое 30 мм, правое 10 мм, верхнее и нижнее 20 мм; текст с абзацным отступом 10 мм, адресат — блоком на 90 мм от левого поля, номер страницы — со второй страницы вверху посередине.

4.1. Пример

```
---
doc:
  title: "Щодо графіка технічної підтримки"
  language: uk-UA
styles: [std/styles/letter]
params:
  номер: "12/3"
  дата: 2026-10-16
  відправник: {назва: "ТОВ «Виконавець»", адреса: "вул. Сумська, 1, м. Харків, 61000",
телефон: "+380 57 000 00 00", код: "00000000", керівник: "Петро ПЕТРЕНКО"}
  адресат: {посада: "Директору", назва: "ТОВ «Замовник»", особа: "Івану ІВАНЕНКУ"}
---
|.LetterheadName: |(відправник.назва)
|.LetterheadDetails: |(відправник.адреса), тел. |(відправник.телефон)
|.LetterheadDetails: Код ЄДРПОУ |(відправник.код)
|.OutgoingNumber: Вих. № |(номер) від |(fmt_date(дата, "short"))

|.Recipient: |(адресат.посада) |(адресат.назва)
|.Recipient: |(адресат.особа)

|.LetterSubject: |(doc.title)

|.Salutation: Шановний пане Іване!

Повідомляємо, що роботи з налаштування обладнання завершено, акт приймання-передачі
підписано сторонами.

Пропонуємо погодити графік технічної підтримки на листопад — грудень 2026 року. Проєкт
графіка додається.

|.AttachmentLabel: Додаток: проєкт графіка на 1 арк. в 1 прим.

|table(style="TableParties", header=0, cols=["60%", "40%"]):
|: Директор |& |(відправник.керівник)
|end table

|.Executor: Олена Коваленко +380 57 000 00 01
```

4.2. Части письма

Часть	Стиль	Замечание
название и реквизиты отправителя	`LetterheadName`, `LetterheadDetails`	в левой части, строки до 73 мм
номер и дата	`OutgoingNumber`	«Вих. № … від …»; дата — `fmt_date(дата, "short")`
адресат	`Recipient`	каждая строка — отдельный абзац этого стиля
заголовок к тексту	`LetterSubject`	«Щодо …», без точки
обращение	`Salutation`	по центру

Часть	Стиль	Замечание
текст	без стиля	обычные абзацы получают `BodyIndented` — набор привязывает к нему текст
приложения	`AttachmentLabel`	
подпись	таблица `TableParties`	должность слева, имя справа на одной линии
исполнитель	`Executor`	мелким шрифтом в конце

Первая страница письма — стиль страницы `LetterFirst` без номера; следующие — `Portrait` с номером вверху. Это задаёт набор, в тексте ничего писать не нужно.

4.3. Свои отличия

Если в организации принят другой бланк, не правьте набор комплекта: сделайте свой модуль поверх него и подключите его вместо набора (страница «Стили»):

```
---
kind: styles
extends: std/styles/letter
paragraphs:
  LetterheadName: {size: 13pt, color: "#1F4E79"}
---

---
doc: {title: "Лист", language: uk-UA}
styles: [styles/бланк.s.prml]
---
|.LetterheadName: ТОВ «Приклад»

Текст листа.
```

Логотип на бланке — `|img("assets/логотип.svg", style="LogoSmall")` в начале письма; размеры логотипа и иллюстраций (`LogoSmall`, `Logo`, `Illustration` …) есть в наборе `business`.

5. Техническая документация

Руководства, инструкции, технические описания оформляются набором стилей `std/styles/technical` (или `report` — для отчётов по ДСТУ 3008). Ниже — средства, которые нужны именно техническим документам: предупреждения, варианты для разных исполнений изделия, сокращения, указатель, рисунки.

5.1. Предупреждения и примечания

`|!("вид")` — смысловая разметка: автор называет вид, а сигнальное слово на языке документа, цвет и оформление берутся из стилей.

Вид	Слово (uk)	Когда
`danger`	НЕБЕЗПЕКА	неминуемая угроза жизни
`warning`	ПОПЕРЕДЖЕННЯ	возможная угроза жизни или тяжёлая травма
`caution`	ОБЕРЕЖНО	возможная лёгкая травма
`notice`	УВАГА	возможное повреждение имущества
`note`	Примітка	пояснение
`tip`	Порада	рекомендация

```
|!("warning", hazard="Висока напруга", consequence="Ураження електричним струмом"):
Перед обслуговуванням від'єднайте пристрій від мережі.

|- Вимкніть автоматичний вимикач.
|- Перевірте відсутність напруги.
|end!

|!("notice") Не встановлюйте пристрій поблизу джерел тепла.

|!("note") Значення наведено для температури 20 °C.
|!("note") Допуск — ±5 %.
```

- Строчная форма `|!("вид") текст` — блок из одного абзаца; окружение `|!(…):` … `|end!` — из нескольких абзацев и списков.
- У `danger` и `warning` нужен `hazard=` (вид и источник опасности); без него проверка предупредит.
- Несколько `note` подряд объединяются в группу «Примітки:» с нумерацией.

5.2. Варианты для разных исполнений

Один исходный текст обслуживает несколько исполнений изделия. Свойства исполнения перечисляются в параметре `features`, а условные части помечаются `when=`:

```
---
doc: {title: "Настанова з експлуатації", language: uk-UA}
styles: [std/styles/technical]
params:
  модель: "X300"
  features: [wifi]
---
|< Підключення
|> Під'єднайте кабель живлення.
|>(when=has("wifi")) Підключіть пристрій до мережі Wi-Fi.
|>(when=has("bluetooth")) Увімкніть Bluetooth на телефоні.
|table(header=1, cols=["*", "30%"]):
```

```
|: Параметр |& Значення
|: Напруга |& 230|~В
|:(when=модель == "X300") Потужність |& 2,2|~кВт
|end table
|if(has("wifi")):
|< Бездротова мережа
|> Пристрій підтримує стандарт 802.11n.
|endif
```

- `when=` есть у любой блоковой команды, окружения и строки таблицы; `|if(…):` … `|endif` — у произвольной группы блоков.
- `has("wifi")` истинно, если `wifi` есть в `features`. Набор свойств задают заголовок документа, проект или цель сборки проекта (`params: {features: [wifi, bluetooth]}` в `targets`): так из одного текста собираются настановы всех исполнений. Параметр командной строки `--param` списков не принимает.
- Удалённые части не попадают в ODT; нумерация, оглавление и ссылки считаются по оставшемуся тексту. Ссылка на удалённую часть — ошибка.
- `primula build настанова.prml --review` собирает все варианты сразу и помечает каждую условную часть её условием (зелёным — включена в текущем исполнении, красным — нет), а на страницы ставит водяной знак «НЕ ДЛЯ ВИПУСКУ». Так проверяют весь текст одним файлом.

5.3. Сокращения

```
|abbr("ПЗ", "програмне забезпечення") встановлюється з диска.
Оновлення |abbr("ПЗ") виконується автоматично.
|list_of("abbr", title="ПЕРЕЛІК СКОРОЧЕНЬ")
```

При первом употреблении выводится «програмне забезпечення (ПЗ)», дальше — только «ПЗ». Падеж расшифровки в этом месте — `text="програмного забезпечення"`. `|list_of("abbr")` строит перечень сокращений таблицей.

5.4. Указатель, оглавление, перечни

```
|toc(max_level=2, title="ЗМІСТ")
|< Живлення
Напруга|index("напруга живлення") подається через адаптер.
|list_of("index", title="ПОКАЖЧИК")
```

Оглавление, перечни рисунков и таблиц (`|list_of("figure")`, `|list_of("table")`) и алфавитный указатель — обновляемые указатели Writer; номера страниц в них заполняет Writer или транслятор при сборке PDF.

5.5. Рисунки

```
|figure(svg='<svg xmlns="http://www.w3.org/2000/svg" width="40mm" height="20mm"><rect
x="1" y="1" width="38" height="18" fill="none" stroke="black"/></svg>', id="рис-
схема"):
Схема підключення
|end figure
Див. |@("рис-схема", "label").
```

- `|figure(путь или svg=…, id=…):` … `|end figure` — нумерованный рисунок; тело — подпись («Рисунок 1 — Схема підключення»). `|img(…)` — изображение без номера и подписи.

- Путь к файлу — относительно документа: `|figure("assets/схема.svg", width="120mm")`. Форматы — PNG, JPEG, SVG.
- `svg=` — изображение из строки SVG, вычисленной кодом (график, шкала, QR-код из функции модуля).
- Размер — `width=` или `height=`; размеры по стилю — `style="Illustration"` (наборы `business`, `letter`).

5.6. Текст программ и интерфейс

Набор `technical` даёт стили для элементов интерфейса и команд:

```
---
doc: {title: "Інструкція", language: uk-UA}
styles: [std/styles/technical]
---
|list("StepList"):
|# Натисніть |.Key{Ctrl+S}.
|# Виберіть |.MenuItem{Файл → Зберегти} і вкажіть файл |.FileName{звіт.odt}.
|end list
|.StepResult: Файл буде збережено.

|.Preformatted: primula check звіт.prml
```

Символьные `Key`, `MenuItem`, `FileName`, `Command`, `Variable`; абзацные `Preformatted`, `ProgramOutput`, `StepResult`, `DefinitionTerm`/`DefinitionText`; список шагов `StepList`; таблица `TableRevisions` для листа регистрации изменений. Полный перечень — приложение «Стили комплекта». Текст, который нужно сохранить буква в букву (листинг с символами `|`), пишут в `|verbatim:` … `|end`.

5.7. Общие фрагменты и ключи

Повторяющиеся тексты (общие меры безопасности, гарантия) выносят во фрагменты `*.f.prml` и подключают `|include`; в проекте их можно подменять по исполнению через карту ключей `keys:` — страница «Проект». Полный пример — `examples/03-manual` и `examples/06-safety`.

6. Проект из нескольких документов

Когда документов несколько — договор, акты, счета, письма одной компании, — общие данные, стили и повторяющиеся тексты собирают в проект. Проект — каталог с файлом `project.p.prml`. Транслятор находит его сам, поднимаясь от документа вверх (до каталога с `.git`).

```
договори-2026/
  project.p.prml        параметры, стили, ключи, цели сборки
  договір.prml
  акт.prml
  fragments/            тексты для |include (*.f.prml)
  lib/                  макросы и функции для |import (*.m.prml)
  styles/               свой модуль стилей (*.s.prml)
  data/                 CSV, YAML, JSON
```

6.1. Файл проекта

```
---
kind: project
name: "Договори 2026"
language: uk-UA
styles: [styles/компанія.s.prml]
doc_defaults: {organization: "ТОВ «Виконавець»", creator: "Відділ договорів"}
params:
  місто: "м. Харків"
  виконавець: {назва: "ТОВ «Виконавець»", код: "00000000", директор: "П. П. Петренко"}
  ставка_пдв: 20
targets:
  - {source: договір.prml, output: out/договір.odt, pdf: true}
---
```

- `params:` — общие параметры. Документ видит их по имени (`|(місто)`) и как `|(project.місто)`. Параметр документа с тем же именем заменяет значение проекта для этого документа.
- `doc_defaults:` — метаданные ODT по умолчанию для всех документов; ключ `doc:` документа заменяет их поштучно.
- `styles:` — модули стилей проекта; они подключаются ко всем документам.
- `targets:` — цели пакетной сборки.

Свой модуль стилей проекта построен на наборе комплекта (страница «Стили»):

```
---
kind: styles
extends: std/styles/business
paragraphs:
  Реквізити: {parent: Details, keep_together: true}
---
```

6.2. Документ проекта

```
---
doc: {title: "Договір про надання послуг"}
---
|(виконавець.назва), |(місто). Ставка ПДВ — |(percent(ставка_пдв)).

|.Реквізити: Код ЄДРПОУ |(виконавець.код)
```

Документ не называет ни стилей, ни языка: всё приходит из проекта.

6.3. Фрагменты: общий текст

Фрагмент `*.f.prml` — кусок текста для вставки. Параметры фрагмента объявляются в его заголовке и передаются аргументами `|include`:

```
---
params: {місяців: 12}
---
|> Гарантійний строк становить |(місяців) |(plural(місяців, "місяць", "місяці",
"місяців")).

|< Гарантія
|include("fragments/гарантія.f.prml", місяців=24)
```

- Путь — от файла с командой; расширение можно опустить (`fragments/гарантія`).
- Имена, определённые во фрагменте, документу не видны; чтобы передать значение наружу, фрагмент экспортирует его: `|export(строк=…)`, функцию — декоратором `@export` над `def`, готовый макрос — `|export("підпис")` (страница «Код»).
- Часть фрагмента помечают `|part("назва"):` … `|end part` и вставляют `|include("файл", part="назва")`.

6.4. Модули: макросы и функции

Модуль `*.m.prml` содержит только определения — макросы `|def`, функции и данные в `|code:`. Текста в нём нет.

```
|def підпис(сторона, особа):
|.SignatureRole: |(сторона)
|.SignatureLine: _____________ |(особа)
|enddef

|import("lib/підписи")
|підписи.підпис("Виконавець", виконавець.директор)
```

После `|import("lib/підписи")` определения доступны как `підписи.имя`; `|import("lib/підписи", "підпис")` — без префикса, `alias="п"` — под коротким именем. Экспортировать из модуля ничего не нужно: открыты все имена, кроме начинающихся с `_`. `|import` документа действует и в его фрагментах. Модуль вычисляется один раз на сборку.

6.4.1. Общая библиотека нескольких проектов

Путь `|import` не выходит за корень проекта, но символическая ссылка, созданная внутри проекта, может вести к общему каталогу (решение 136):

```
ln -s ../shared-lib lib     # в корне проекта
```

После этого `|import("lib/ціни")` берёт модуль из `../shared-lib`. Сборка выдаёт предупреждение P2639 с именем ссылки и её целью: каталог вне проекта не попадает в архив исходников, поэтому храните его вместе с проектами (или в том же репозитории). Пути внутри библиотеки отсчитываются от `lib/…`, и `../` в тексте по-прежнему не выводит за корень. В отчёте и сообщениях файл записывается как `lib/ціни.m.prml`.

6.5. Данные из файлов

```
номер,дата,сума
1,2026-10-31,5400.00
2,2026-11-30,5400.00

|code:
акти = load_csv("data/акти.csv", types={"номер": "decimal", "дата": "datetime",
"сума": "money"})
|end
|for(a in акти):
|- Акт № |(a.номер) від |(fmt_date(a.дата, "short")) на |(money(a.сума, "UAH")).
|endfor
```

Без `types=` все значения CSV — строки (коды с ведущими нулями не портятся). YAML и JSON читают `load_yaml` и `load_json`; отображения становятся записями с доступом через точку.

Таблицы LibreOffice Calc и Excel читают `load_ods` и `load_xlsx` (решение 138): лист — `sheet="Позиції"` или номер с 1, диапазон — `range="A1:D20"`, первая строка — имена полей (`header=False` — поля `A`, `B`, …), `types` — как у CSV.

```
|code:
позиції = load_ods("data/позиції.ods", sheet="Позиції", types={"ціна": "money"})
|end
```

Формулы не вычисляются: берётся результат, сохранённый в файле, — сохраните таблицу в Calc или Excel после правки. Числа — `decimal` до 15 значащих цифр, даты — `datetime`; ошибка в ячейке (`#DIV/0!`) или формула без результата останавливают сборку с именем листа и ячейки. Файлы `.xls` сначала сохраните как `.xlsx` или `.ods`.

Весь текстовый файл одной строкой возвращает `load_text` (решение 140): UTF-8, концы строк — LF, без разбора. Так удобно держать вне документа SVG рисунка, знака или схемы и вставлять его через `svg=`:

```
|img(svg=load_text("img/знак.svg"), width="12mm")
```

6.6. Сборка проекта

```
primula build project.p.prml --pdf      # все цели проекта
primula check project.p.prml            # проверка всех целей без записи
primula build договір.prml              # один документ (с параметрами и стилями
проекта)
primula check project.p.prml --dump-project   # описание проекта в JSON
```

Цели можно порождать кодом: секция кода файла проекта (после заголовка) читает данные и вызывает `build()` — например, по акту на каждую строку CSV:

```
---
kind: project
---
for row in load_csv("data/акти.csv", types={"номер": "decimal", "сума": "money"}):
    build("акт.prml", params={"номер": row.номер, "сума": row.сума}, output=f"out/акт-
{row.номер}.odt", pdf=True)
```

Ошибка одной цели не останавливает остальные. Полный пример с договором, актами из CSV, картой ключей и сводным пакетом — `examples/07-project`.

6.7. Экспорт в Markdown, HTML, DOCX

`primula export` записывает вычисленный документ в другие форматы через pandoc: номера пунктов, ссылки, сноски, таблицы и подписи сохраняются, разметка страниц (колонтитулы, колонки, номера страниц) — нет.

```
primula export договір.prml                    # договір.md
primula export договір.prml --to html          # один файл HTML с изображениями
primula export договір.prml --to docx          # стили абзацев — по именам (custom-
style)
primula export договір.prml --to json          # AST pandoc, pandoc не нужен
```

6.8. Сводный документ

Несколько документов собираются в один ODT командой `|document`:

```
|document("договір.prml", scope="договір")
|document("акт.prml", scope="акт1", params={"номер": 1})
```

Каждая часть начинается с новой страницы, нумеруется в своей области и вычисляется отдельно; оглавление и нумерация страниц — общие. Ссылка на другую часть — по полному ID: `|@("акт1:перелік")`.

6.9. Ссылки на другой документ

Акт может ссылаться на пункт договора, если договор назван в `ref_documents:` проекта, а акт перечисляет его в `references:` своего заголовка: `|@("оплата", "label")` в акте даст «п. 2.2» договора. Текст ссылки берётся из реестра договора, который транслятор обновляет при сборке.

6.10. Карта ключей

Ключи `keys:` проекта дают ресурсам и фрагментам имена: `|img(key="логотип")`, `|include(key="умови.оплата")`. Цель сборки может подменить ключ (`keys:` у цели) — так один текст собирается с разными логотипами или вариантами условий.

6.11. Проект заказа

Для серии заказов по одному договору делают проект в подкаталоге с `extends: ../../project.p.prml`: он наследует параметры, стили, ключи и документы базового проекта и добавляет свои данные заказа. В Studio — «Файл → Создать → Проект заказа».

7. Стили

Оформление документа задают стили, а не текст: шрифт, отступы, интервалы, рамки таблиц, поля страниц. В тексте пишется только *что* это — заголовок, пункт, реквизиты, — а *как* это выглядит, решает модуль стилей. Поэтому один и тот же документ можно напечатать по-другому, подключив другой набор, а правка оформления не трогает текст.

7.1. Выбор набора

В комплекте транслятора — наборы стилей для типичных документов:

Набор	Для чего	Основа
`std/styles/default`	подключается сам, если стили не названы	—
`std/styles/business`	договоры, акты, счета, протоколы; ДСТУ 4163:2020	`default`
`std/styles/letter`	служебные письма; ДСТУ 4163:2020	`business`
`std/styles/technical`	руководства, инструкции, технические описания	`default`
`std/styles/report`	отчёты в сфере науки и техники; ДСТУ 3008:2015	`default`
`std/styles/compact`	памятки, справки, шпаргалки	`default`

Документ называет набор одной строкой заголовка:

```
---
doc: {title: "Акт", language: uk-UA}
styles: [std/styles/business]
---
|.DocumentTitle: АКТ № 14
```

Основу (`default`) перечислять не нужно: набор сам подключает её. Стили каждого набора, их свойства и шрифт по умолчанию — в приложении «Стили комплекта»; образцы — документы `examples/08-styles` (их можно собрать и посмотреть PDF).

7.2. Применение стилей в тексте

Запись	Что делает
`|.DatePlace: м. Харків`	абзацный стиль для абзаца этой строки
`|.Note:` … `|end.Note`	абзацный стиль для нескольких абзацев (окружение)
`|.DefinedTerm{Замовник}`	символьный стиль для фрагмента внутри строки
`|*{…}`, `|/{…}`, `|`{…}`	полужирный, курсив, моноширинный — это тоже символьные стили (`Strong`, `Emphasis`, `Code`)
`|table(style="TableCompact")`	стиль таблицы
`|list("DashList")`	стиль следующего списка

Имя стиля в `|.Имя` — идентификатор: оно не переводится и не меняется при вычитке текста. Неизвестное имя — ошибка с позицией.

Заголовки, пункты, списки, таблицы и подписи получают стили автоматически — их задаёт привязка конструкций в модуле стилей (`Heading1`…`Heading6`, `Clause`, `BulletList`, `TableGrid`, `TableCaption` …). Поменять стиль конструкции до конца окружения можно командой `|set`:

```
|set(list="DashList")
|- перший пункт;
|- другий пункт.
```

Прямое оформление `|fmt(bold=True){…}` тоже есть, но даёт предупреждение: разовое оформление теряется при смене набора, а одинаковые места оформляются по-разному. Если оформление повторяется — заведите стиль.

7.3. Свой модуль стилей

Не правьте наборы комплекта — сделайте свой модуль поверх набора. Он содержит только отличия и называет основу ключом `extends`:

```
---
kind: styles
extends: std/styles/business
defaults: {font: Roboto, size: 11pt}
paragraphs:
  Body:        {line_height: "120%"}
  Реквізити:   {parent: Details, keep_together: true}
characters:
  Акцент:      {bold: true, color: "#1F4E79"}
tables:
  TableServices: {header_background: "#DDE6F0"}
---

---
doc: {title: "Договір", language: uk-UA}
styles: [styles/компанія.s.prml]
---
Текст з |.Акцент{акцентом}.

|.Реквізити: Код ЄДРПОУ 00000000
```

- Документ (или проект) называет только свой модуль: основа и её основа (`business` → `default`) подключаются сами.
- Свойство стиля с тем же именем дополняет стиль набора: `Body: {line_height: "120%"}` меняет только интервал, остальное остаётся от набора.
- Новый стиль наследует существующий через `parent`.
- Шрифт всего документа меняется одной строкой `defaults: {font: …}`. Шрифт должен быть установлен там, где документ открывают или собирают PDF.

Группы стилей в модуле: `paragraphs` (абзацные), `characters` (символьные), `tables`, `lists`, `blocks`, `frames` (размеры и размещение изображений), `pages` (страницы). Свойства — `font`, `size`, `bold`, `align`, `line_height`, `space_before`, `space_after`, `left`, `first_line`, `keep_with_next` и другие; полный перечень — раздел 11.3 спецификации. Подсказки по свойствам и стилям даёт Studio (наведение мыши, `Ctrl+Space`).

7.4. Слои

Если стили названы в нескольких местах, они накладываются слоями, от раннего к позднему:

1. модули из `styles:` проекта;
2. модули, переданные при сборке `--style ФАЙЛ`;
3. модули из `styles:` документа;
4. стили, определённые кодом самого документа.

Более поздний слой дополняет и переопределяет ранние. Основа модуля (`extends`) — слой сразу перед ним. Какие слои реально подключены и откуда взялся каждый стиль, показывает панель «Стили» Studio или `primula check документ.prml --dump-styles`.

7.5. Страницы и колонтитулы

Поля и размер страницы — стиль страницы (`pages:` модуля, по умолчанию `Portrait`; в наборах есть `Landscape`). Колонтитулы задаются в коде модуля стилей или в документе:

```
|footer("Portrait"):
|.Footer: Договір № П-2026/017 — сторінка |field.page з |field.pages
|end footer
```

Смена страницы с этого места — `|page(style="Landscape")`, разрыв — `|newpage`.

7.6. Переход с ODT Markdown Kit

Деловой набор Kit 1.2 (`business-styles.yaml`) перенесён в `std/styles/business` и `default` под теми же именами, кроме перечисленных ниже:

Kit	Primula
`TocTitle`, `TocEntry`	`ContentsTitle`, `Contents1`…`Contents6`
`StrongText`, `EmphasisText`, `UnderlinedText`	`Strong`, `Emphasis`, `Underline` (короткие формы `|*{}`, `|/{}`, `|_{}`)
`DraftPlaceholder`	`Placeholder` (короткая форма `|?{…}`)
`BodyRoboto`, `FontCarlito` и подобные	нет: шрифт задаётся `defaults: {font: …}` своего модуля
`Recipient`, `LetterSubject`, `Salutation`	в наборе `letter`
`Approval`	в `business`, на 100 мм от левого поля (ДСТУ 4163:2020)
`NumberedList` с буквами `a)`	`NumberedList` (`1.`, `1.`, `a)`), `LetterList` — буквы языка документа «а)», `LatinLetterList` — «a)»
пресеты изображений `Logo`, `Illustration` …	стили фреймов с теми же именами: `|img(…, style="Logo")`

Номера пунктов и ссылки в Kit писались текстом; в Primula их ставит транслятор (`|>`, `|@("id")`), а стиль `Clause` применяется к пунктам сам.

8. Сообщения транслятора

Транслятор проверяет документ целиком — от кодировки до ссылок и таблиц — и сообщает обо всём, что нашёл, а не только о первой ошибке. Документ с ошибками не собирается; с предупреждениями — собирается, но предупреждение почти всегда указывает на настоящую проблему.

8.1. Как читать сообщение

```
договір.prml:6:9: error[P2201]: имя 'сумма' не определено
 6 | Сума: |(сумма).
   |         ^^^^^
договір.prml: 1 ошибка, 0 предупреждений
```

- `договір.prml:6:9` — файл, строка, столбец. Ошибка во фрагменте или модуле указывает их файл и место подключения.
- `error` — ошибка, `warning` — предупреждение.
- `P2201` — код. Первая цифра — область: P0 командная строка и файлы, P1 синтаксис, P2 имена и вычисления, P3 стили, P4 структура и ссылки, P5 таблицы, P6 страницы и графика, P7 модули и проект, P8 запись ODT, P9 техническая документация. Полный перечень с текстами — приложение «Сообщения».
- Строка с `^^^` показывает место; строки `= help:` и `= note:` — подсказка и пояснение.

Язык сообщений — `--lang uk|ru|en` или переменная `PRIMULA_LANG`. Для программ и редакторов — `--format json`: список сообщений с полями `code`, `severity`, `message`, `file`, `line`, `column`. Код завершения: 0 — без ошибок, 2 — есть ошибки.

В Studio сообщения проверки собраны в панели; щелчок открывает место в редакторе, F1 на конструкции — её раздел спецификации.

8.2. Предупреждения и выпуск

`--strict` превращает предупреждения в ошибки с тем же кодом. Так собирают окончательную редакцию: незаполненные места `|?{…}`, неоформленные числа и прямое оформление не пройдут.

```
primula build договір.prml --strict --pdf
```

8.3. Частые ошибки

8.3.1. Стиль без текста — P1209

Имя стиля после `|.` — идентификатор, а не текст. Если при правке оно изменилось («Місце-дата» → «Місце і дата»), строка перестаёт быть стилем:

```
|.Місце-дата
```

Верните имя стиля и пишите текст после двоеточия: `|.Місце-дата: м. Харків, 12.10.2026`. Список имён стилей — панель «Стили» Studio или `--dump-styles`.

8.3.2. Имя не определено — P2201

```
Сума: |(сумма).
```

Опечатка в имени параметра или переменной, параметр не объявлен в `params:` документа или проекта, функция модуля без `|import`. Модуль `std/math` в документе подключают явно: `|import("std/math")`.

8.3.3. Команда «съела» слово — P2601

```
|(1)|sellerom
```

Буква сразу после имени команды продолжает имя: `|sellerom` — это команда `sellerom`. Разделитель `|;` заканчивает имя: `|seller|;ом`.

8.3.4. Пробел после черты — P1201

```
Ціна | 100 грн.
```

После `|` должны идти буква, `(` или знак короткой формы. Сама черта в тексте пишется двумя: `||`.

8.3.5. Неверное закрытие — P1218, P1219, P1224

`|end` закрывает ближайшее окружение, но не управляющие конструкции: `|if` закрывается `|endif`, `|for` — `|endfor`, `|def` — `|enddef`. Блок `|code:` закрывается строкой из одного `|end`, без имени (`|end code` — ошибка P1224). Пункт в строчной форме (`|> текст`) окружением не является, и `|end` после него — лишний (P1218).

```
|> Пункт.
|end
```

8.3.6. Значение нельзя вывести — P2603, P2604

`|(…)` выводит строку, число, дату или разметку. `None`, логическое значение, список или запись — ошибка P2603: используйте `str()`, `join_list()` или поле записи.

Дробное число, сумма или дата без функции оформления — предупреждение P2604:

```
|(dec("1.5"))
```

Оформляйте библиотекой: `num(x, 2)`, `money(x, "UAH")`, `fmt_date(d, "long")`.

8.3.7. Неизвестный стиль — P3108

```
|.Невідомий: текст
```

Стиля нет ни в одном подключённом модуле: опечатка или не подключён набор (`styles:`). Какие модули подключены — `--dump-styles` или панель «Стили».

8.3.8. Ссылка и ID — P4301, P4102

```
Див. |@("оплата").
```

Цели с таким `id=` нет: опечатка, цель удалена или исключена условием. Повторный `id=` — ошибка P4102 с позициями обоих мест.

8.3.9. Пропуск уровня — P4201

```
|< Розділ
|<<< Підрозділ
```

Заголовки идут по уровням: после `|<` — `|<<`, а не `|<<<`. Так же со списками и пунктами.

8.3.10. Таблица — P5103, P1226

```
|table(cols=["*", "*"]):
|: а |& б |& в
|end table
```

Число ячеек строки не совпадает с числом столбцов (с учётом `span=`). Пустая ячейка пишется явно: `|&` без текста. Текст внутри таблицы вне строки `|:` — ошибка P1226.

8.3.11. Ключ заголовка переименован — P1120

Ключи `document:` и `documents:` прежних версий переименованы: метаданные — `doc:`, в проекте — `doc_defaults:` и `ref_documents:`.

8.3.12. Заполнитель — P2635

`|?{…}` оставлен в документе. При черновой сборке это предупреждение, при `--strict` — ошибка.

8.4. Если непонятно, что не так

- Проверьте участок отдельно: `primula console документ.prml` открывает консоль вычислителя с именами документа — там можно вычислить выражение (`:help имя` — описание функции).
- `primula check документ.prml --dump-tree` показывает, как транслятор понял структуру: где кончается пункт, что вошло в таблицу.
- В Studio правая кнопка мыши на конструкции даёт подсказку и раздел спецификации.

9. Параллельные тексты

Окружение `|parallel` ставит рядом два или больше текстов — потоки — и выравнивает по горизонтали начала их смысловых блоков — сегментов. Так пишутся двуязычный договор, протокол разногласий («было / стало»), текст с комментарием на полях. Колонки раздела (`|section(cols=2)`) для этого не годятся: они текут независимо. Таблица годится по виду, но в ней нет понятия «пункт и его перевод»: нумерация, ссылки и язык общие на всю таблицу.

Транслятор выводит `|parallel` таблицей без рамок, но знает потоки: каждый поток нумеруется своими счётчиками, вычисляется на своём языке и сверяется с основным потоком.

9.1. Пример

```
---
doc: {title: "Договір / Agreement", identifier: "ДП-2026/017", language: uk-UA}
params:
  ціна: 48000.00
  виконавець: {uk: "ТОВ «Виконавець»", en: "Vykonavets LLC"}
styles: [std/styles/business]
---
|parallel(tracks=[track("uk", lang="uk", title="Українською мовою"), track("en",
lang="en", title="In English")],
          gap="8mm", rule="0.3pt solid #999999", parity="translation"):

|segment(span="all"):
|.DocumentTitle: ДОГОВІР № |(doc.identifier) / AGREEMENT No. |(doc.identifier)
|end segment

|segment(id="subject"):
|track("uk"):
|<(id="subject") Предмет договору
|>(id="services") |(getattr(виконавець, track.id)) надає послуги з розроблення
документації.
|> Ціна послуг — |(money_full(ціна, "UAH")) (див. |@("services", "label")).
|track("en"):
|<(id="subject") Subject of the Agreement
|>(id="services") |(getattr(виконавець, track.id)) provides the services of developing
documentation.
|> The price of the services is |(money_full(ціна, "UAH")) (see |@("services",
"label")).
|end segment

|end parallel
```

В украинской колонке сумма выходит «сорок вісім тисяч гривень 00 копійок», ссылка — «п. 1.1»; в английской — «forty-eight thousand hryvnias 00 kopiykas» и «cl. 1.1». Пункты нумеруются в обеих колонках одинаково.

9.2. Потоки

Поток описывается функцией `track(id, …)` в аргументе `tracks=`:

Аргумент	Что задаёт
`lang`	язык потока: суммы прописью, даты, кавычки, метки ссылок, проверка орфографии в Writer
`title`	название над колонкой; строка названий повторяется на каждой странице
`width`	ширина колонки, как у столбца таблицы: `"30%"`, `"60mm"`, `"*"`

Аргумент	Что задаёт
`primary=True`	основной поток (по умолчанию первый): от него идёт нумерация документа, оглавление, навигация, с ним сверяются остальные
`numbered=False`	поток без нумерации: комментарии, примечания на полях
`optional=True`	поток может не иметь блока в сегменте — ячейка остаётся пустой
`block`	стиль блока для всех ячеек потока (фон, отступы)

Если нужны только языки, хватает сокращения `langs=["uk", "en"]`. Промежуток между колонками — `gap=`, линия — `rule=`.

Параметры документа общие для всех потоков. Значение на языке потока выбирается кодом: в блоке потока доступна запись `track` (`track.id`, `track.lang`), поэтому `|(getattr(виконавець, track.id))` берёт поле `uk` или `en`.

9.3. Сегменты

Сегмент — строка выравнивания: блок каждого потока начинается на одном уровне, конец — по самому длинному блоку. Сегментом обычно делают раздел договора или пункт.

- **Окружение:** `|segment(id="…"):`, затем блоки потоков `|track("uk"):`, `|track("en"):` и `|end segment`. Блок потока кончается на следующем `|track` или на закрытии сегмента.
- **Строка** — для коротких блоков:

```
|parallel(langs=["uk", "en"]):
|segment(id="city") |track("uk"){м. Харків} |track("en"){Kharkiv}
|end parallel
```

- **На всю ширину** — общий текст для всех потоков (название договора, таблица подписей): `|segment(span="all"):` без блоков потоков.

Внутри блока потока пишется обычный текст: заголовки, пункты, списки, таблицы, рисунки, сноски. Нельзя: смена страницы, разделы с колонками, приложения, колонтитулы, оглавление, вложенный `|parallel` (сообщение P5304). Между сегментами допустимы `|newpage` и `|page(…)`: таблица продолжается на новой странице со строкой названий.

Заголовок, оставшийся последним на странице, Writer не переносит к следующему сегменту. Пишите заголовок в одном сегменте с первым пунктом раздела.

9.4. Нумерация, ссылки, сноски

- У каждого потока свои счётчики: при входе в `|parallel` они копируются из документа, после окружения документ продолжает нумерацию основного потока.
- ID внутри потока получает имя потока: `id="services"` в украинском блоке — это `uk.services`, в английском — `en.services`. Поэтому одинаковые `id=` в обоих потоках не конфликтуют.
- `|@("services")` внутри потока ищет цель в своём потоке, затем в основном, затем во всём документе. Цель другого потока пишется полностью: `|@("uk.services")`.
- Сноска неосновного потока получает номер своего потока: в обеих колонках номера одинаковые.
- Заголовки неосновных потоков выглядят как заголовки, но в навигатор Writer и закладки PDF не попадают — там только основной поток.

Оглавление документа берёт заголовки основного потока. Оглавление другого потока — `|toc(track="en")`; название по умолчанию — на языке потока («CONTENTS»):

```
|toc(title="ЗМІСТ")
|toc(track="en")
|parallel(langs=["uk", "en"]):
|segment:
|track("uk"):
|< Предмет договору
|track("en"):
|< Subject of the Agreement
|end segment
|end parallel
```

9.5. Проверка соответствия

Аргумент `parity=` сверяет каждый нумеруемый поток с основным в каждом сегменте:

Профиль	Что проверяется
`structure` (по умолчанию)	есть ли блок каждого обязательного потока; последовательность заголовков, пунктов, элементов списков, таблиц, рисунков, примечаний
`translation`	то же, а также ID, цели ссылок, имена параметров в `|(…)`, числа и адреса в тексте, счётчики потоков в конце
`none`	ничего

Сегмент может задать свой профиль: `|segment(id="idiom", parity="none")` — для вольного перевода. Расхождения — предупреждения P5310–P5315; с `--strict` они становятся ошибками.

```
|parallel(langs=["uk", "en"], parity="translation"):
|segment(id="term") |track("uk"){Строк — 5 днів.} |track("en"){The term is 7 days.}
|end parallel
```

9.6. Перевод: заготовки, подтверждение, актуальность

Когда документ переводится, удобно вести состояние каждого блока:

1. **Заготовки.** `primula parallel договір.prml --stub --track en` дописывает в сегменты без английского блока копию структуры основного блока с заполнителями `|?{…}`. Заполнители — предупреждения, с `--strict` — ошибки, поэтому незаполненный перевод не уйдёт в выпуск.
2. **Подтверждение.** Проверенный перевод фиксируется: `primula parallel договір.prml --stamp --track en --status reviewed` записывает в блоки `status="reviewed"` и `rev="…/…"` — хеши текста основного блока и самого перевода. `--segment ID` подтверждает только названные сегменты.
3. **Изменение оригинала.** Если после подтверждения изменён украинский текст, `check` выдаёт P5320 «перевод устарел»; если изменён сам перевод — P5321. Предупреждение остаётся, пока перевод не исправлен и не подтверждён снова.
4. **Перед выпуском.** `primula parallel договір.prml --check-status reviewed` завершается ошибкой, если какой-то блок не в этом состоянии или устарел.

В Studio те же действия — в меню «Сборка → Параллельные потоки» и в контекстном меню редактора («Подтвердить перевод сегмента …» для блока под курсором). Панель «Структура» показывает сегменты и блоки потоков со статусом и отмечает устаревшие.

Хеш считается по тексту блока без комментариев и лишних пробелов, поэтому переформатирование не делает перевод устаревшим. Выражения `|(…)` входят в хеш как записаны: смена значения параметра перевод не устаревает — значения общие для потоков.

9.7. Протокол разногласий

Три потока: редакция поставщика (необязательная — нового пункта в ней нет), редакция покупателя (основной поток) и комментарий юриста (без нумерации, узкая колонка). Номера пунктов договора пишутся явно:

```
|parallel(tracks=[track("old", title="Редакція Постачальника", optional=True),
                  track("new", title="Редакція Покупця", primary=True),
                  track("note", title="Коментар", width="30%", numbered=False,
optional=True)]):
|segment(id="p-4-2"):
|track("old"):
|*{4.2.} Оплата здійснюється протягом 5 банківських днів.
|track("new"):
|*{4.2.} Оплата здійснюється протягом 30 календарних днів.
|track("note"):
|.SmallPrint: Відстрочку погоджено листом від 12.10.2026.
|end segment
|segment(id="p-4-3"):
|track("new"):
|*{4.3.} Покупець має право на дострокову оплату.
|end segment
|end parallel
```

Полные примеры — `examples/09-parallel/agreement.prml` (двуязычный договор с подтверждёнными переводами) и `protocol.prml`.

9.8. Стили

Таблица вывода — стиль `ParallelTable` (без рамок), строка названий потоков — `ParallelHeader`; оба есть во всех наборах комплекта (приложение стили комплекта). Свои имена задаются в модуле стилей: `structure_styles(parallel_table="…", parallel_header="…")` или аргументом `style=` у `|parallel`.

10. Код: диалект Python

Код Primula пишется как Python: выражения `|(…)`, аргументы команд, блоки `|code:`, функции и макросы. Синтаксис — подмножество Python 3.11, но смысл кое-где другой: числа десятичные, глобальные имена неизменяемые, вывода (`print`) нет. Эта страница перечисляет то, что надо знать автору, который знает Python; что работает как в Python, здесь не повторяется.

Полный пример с таблицами «выражение → Primula → Python 3» — `examples/10-code` (документ `code.prml`, фрагменты по темам, файлы ошибок `forbidden.prml` и `mistakes.prml`). Пример показывает исходный текст: код и определения в PDF не печатаются, поэтому PDF читается вместе с исходником. Функции библиотеки — в приложении «Функции выражений».

10.1. Где пишется код

Запись	Назначение
`|(сума * 2)`	вывести значение в текст
`|table(cols=["*", "30mm"])`	аргументы команд — выражения
`|if(n == 1){день}|else{дні}`, `|for(p in позиції){…}`	условие и цикл; тело — разметка, а не код
`|code:` … `|end`	определения, функции, расчёты
`|let(n=5)`	определение посреди текста
`|def підпис(роль):` … `|enddef`	макрос: функция, тело которой — разметка

10.2. Числа: decimal вместо int и float

Целое число и число с точкой без экспоненты — `decimal`: точное десятичное число до 64 значащих цифр, которое помнит количество знаков после запятой. Отдельного `int` нет: `type(5)` — `"decimal"`. Двоичный `float` получается только из литерала с экспонентой (`1.5e3`) или `float(x)`; `decimal` вместе с `float` даёт `float`.

Выражение	Primula	Python 3
`0.1 + 0.2 == 0.3`	`True`	`False`
`2.20 * 3`	`6.60`	`6.6000000000000005`
`7 / 2`	`3.5` (`decimal`)	`3.5` (`float`)
`round(2.5)`	`3`	`2`
`round(2.675e0, 2)`	`2.68`	`2.67`
`10 ** 70`	ошибка P2309	целое из 71 цифры
`True + 1`	ошибка P2304	`2`

Округление одно для всех чисел — половина от нуля, `float` округляется по своей десятичной записи. Деньги — только `decimal`: денежные функции не принимают `float`.

```
Ціна: |(money(2.20 * 3, "UAH")), разом із ПДВ |(money(round(2.20 * 3 * 1.2, 2),
"UAH")).
```

10.3. Строки

Строка — символы Unicode, как в Python 3: `len("Київ") == 4`. Отличия:

- есть только методы из списка спецификации (раздел 12.10): `split`, `join`, `strip`, `replace`, `startswith`, `find`, `upper`, `lower`, `title`, `zfill`, `format` и некоторые другие; `center`, `partition`, `encode` — ошибка P2313;
- f-строки — по синтаксису Python 3.11: внутри фигурных скобок нельзя кавычки того же вида, что вокруг строки (P1208); пишите `f"{d['k']}"`;
- `"{0.name}".format(r)` — ошибка P2330; поле записи читается по ключу: `"{0[name]}"`;
- спецификация формата (`f"{x:,.2f}"`) — английская и языка документа не знает; для текста есть `num()`, `money()`, `fmt_date()`.

```
|code:
сума = 1234.5
|end
Python-формат: |(f"{сума:,.2f}"); формат документа: |(num(сума, 2)).
```

10.4. Списки, кортежи, словари

Как в Python, включая генераторы списков и словарей и распаковку. Отличия:

- множеств нет: `{1, 2}` — ошибка P2101; уникальность — словарём или `not in`;
- операторов `|`, `&`, `^`, `<<`, `~` нет; словари объединяют `{**a, **b}`;
- `is` — только с `None` (P2103);
- ключ словаря — неизменяемое значение; `float` и длина ключом быть не могут (P2317); `5` и `5.00` — один ключ;
- `range()` и `reversed()` возвращают список;
- `lambda` нет: функция для `key=` — обычный `def`.

```
|code:
def мінус(x):
    return -x
порядок = sorted([3, 1, 2], key=мінус)
|end
|(join_list([str(x) for x in порядок]))
```

10.5. Записи

Запись (`record`) — неизменяемое значение с полями: `r.назва` или `r["назва"]`. Записями становятся параметры заголовка YAML, данные проекта, результаты `load_csv`, `load_yaml`, `load_json`. В Python похож `namedtuple`, но запись — не последовательность:

- `"назва" in r` и `len(r)` — ошибки; поле проверяют `hasattr(r, "назва")`;
- отсутствующее поле — ошибка P2314; `getattr(r, "ціна", 0)` даёт значение по умолчанию;
- новая запись с изменённым полем — `r.replace(кількість=4)`; в словарь — `dict(r)`.

```
---
doc: {language: uk-UA}
params:
  сторона: {назва: "ТОВ «Альфа»", код: "12345678"}
---
|(сторона.назва), ІПН: |(getattr(сторона, "ІПН", None) or "—").
```

YAML читается по правилам Primula: `15.00` — `decimal` с двумя знаками, дата без кавычек — `datetime`.

10.6. Даты и длины

Дата и момент — один встроенный тип `datetime`, без `import`; точность — секунда, поясов нет.

- `дата + 30`, `дата - 30` — сдвиг на сутки; `дата - дата` — число суток (`decimal`), а не `timedelta`;
- `hours(36)`, `weeks(2)` — числа суток: `дата + hours(36)`; календарные месяцы — `add_months(дата, 1)` (31 января + 1 месяц = 28 февраля);
- `weekday` — атрибут от 1 (понедельник) до 7, а не метод от 0;
- в текст дата выводится `fmt_date(дата, "contract")`.

Длина — число с единицей: `10*mm` или `"10mm"`; константы `mm`, `cm`, `pt`, `inch` (не `in`). Результат — в единице левого операнда, сравнение — с допуском 0,001 мм.

```
|code:
підписано = date(2026, 10, 12)
оплата = підписано + 10
|end
Оплата до |(fmt_date(оплата, "contract")) — через |(оплата - підписано) днів, |
(fmt_date(add_months(підписано, 1), "long")) — через місяць.
```

10.7. Функции

`def` с умолчаниями, `*args`, `**kwargs`, параметрами только по имени, рекурсией, вложенными функциями и замыканиями; внутри функции переменные изменяемы, есть `while`. Отличия:

- нет `lambda`, генераторов (`yield`), `global`, `nonlocal`; декоратор единственный — `@export` (раздел «Модули и фрагменты»);
- значения по умолчанию вычисляются один раз и замораживаются: ловушка Python `def f(x, acc=[])` здесь — ошибка P2318 при первом `append`;
- глобальный список, переданный в функцию, тоже заморожен: строится новый список;
- вместо предела рекурсии Python — лимиты: глубина 500, 20 с на одно вычисление, 10 000 000 элементов.

```
---
doc: {language: uk-UA}
params:
  позиції:
    - {назва: "Папір", кількість: 20, ціна: 120.50}
    - {назва: "Ручки", кількість: 100, ціна: 15.00}
---
|code:
def рядки(позиції):
    result = []
    for i, p in enumerate(позиції, 1):
        result.append(record(n=i, назва=p.назва, сума=p.кількість * p.ціна))
    return result
|end
|table(cols=["10mm", "*", "35mm"]):
|: № |& Найменування |& Сума, грн
|for(p in рядки(позиції)):
|: |(p.n) |& |(p.назва) |& |(num(p.сума, 2))
```

```
|endfor
|end table
```

10.8. Глобальные имена: ленивые и неизменяемые

Это второе большое отличие после чисел. Python выполняет модуль сверху вниз и позволяет переприсвоить имя. В Primula определения верхнего уровня — `x = …`, `def`, `|let`, параметры заголовка — собираются со всего файла заранее, вычисляются при первом обращении и замораживаются.

- Порядок в тексте не важен: имя можно использовать выше его определения.
- Повторное определение — ошибка P2203, `x += 1` на верхнем уровне — P2105, цикл определений — P2204. Параметр заголовка тоже нельзя переопределить в коде.
- Определение, которое никто не читает, не вычисляется — и его ошибка не проявится, пока имя не понадобится.
- Остальные операторы верхнего уровня (`for`, `if`, вызовы) — действия: выполняются после сбора определений, а имена внутри них локальны (предупреждение P2501).
- Условное значение — условное выражение: `ставка = 20 if платник_пдв else 0`.
- Накопление — генератор списка или функция, а не `append` в цикле верхнего уровня (P2318).

```
|code:
разом = ціна * кількість
ціна = 250.00
кількість = 4
ставка = 20 if разом > 500 else 0
|end
Разом |(num(разом, 2)), ПДВ |(ставка) %.

|code:
x = 1
x = 2
|end
```

10.9. Модули и фрагменты

Модуль (`.m.prml`, `|import`) содержит только определения. Все его имена без `_` в начале открыты, экспортировать ничего не надо; имя с `_` закрыто по-настоящему (`модуль._имя` — ошибка).

```
|code:
_ставка = 20

def з_пдв(сума, ставка=_ставка):
    return round(сума * (100 + ставка) / 100, 2)
|end
```

Подключение	Обращение
`|import("lib/ціни")`	`ціни.з_пдв(…)`, макрос — `|ціни.ім'я(…)`
`|import("lib/ціни", "з_пдв")`	`з_пдв(…)` без префикса
`|import("lib/ціни", alias="ц")`	`ц.з_пдв(…)` (`as` — ключевое слово Python)

`|import` документа действует и во фрагментах, которые документ подключает; `|import` внутри фрагмента — только в этом фрагменте. Путь пишется от файла с командой.

```
|import("lib/ціни")
Разом із ПДВ: |(num(ціни.з_пдв(1000.00), 2)) грн.
```

Фрагмент (`.f.prml`, `|include`) — текст документа со своими определениями, и они закрыты: один фрагмент можно подключить дважды без конфликта. Документ видит только экспортированное (решение 135):

Запись во фрагменте	Что экспортируется
`@export` над `def f(…)` верхнего уровня `|code:`	функция под своим именем
`@export(name="g")` над `def _f(…)`	функция под именем g; `_f` остаётся закрытой
`|export("підпис", "f")`	уже определённые имена: макросы `|def`, функции, `|let`
`|let(x=…, export=True)`, `|export(x=…)`	новое значение

```
---
kind: fragment
---
|code:
@export
def знижка(сума, відсоток):
    return round(сума * (100 - відсоток) / 100, 2)
|end
|def рядок(назва, сума):
|(назва) — |(num(сума, 2)) грн
|enddef
|export("рядок")

|include("fragments/знижки")
|рядок("Папір зі знижкою", знижка(2410.00, 5))
```

Если два фрагмента экспортируют одно имя, конфликт решается при подключении: `|include(…, ns="f")` кладёт экспорт в `f.*`, `rename={…}` переименовывает. `@export` вне фрагмента — ошибка P2610, над вложенной функцией — P2106, в модуле — предупреждение P2638 (там всё и так открыто). Функции, которыми пользуются несколько документов, место в модуле; из фрагмента экспортируют то, что относится к его тексту.

10.10. Значения в тексте и макросы

Вывода нет: значение попадает в документ только выражением `|(…)`, и тип решает, как.

- Строка, число, дата, разметка (`content`) вставляются. Дробное число или дата без оформления — предупреждение P2604: оформляйте `num()`, `money()`, `fmt_date()`.
- `None`, `bool`, список, словарь, запись — ошибка P2603, а не «None» или «[1, 2]», как дал бы `str()`. Необязательное значение — `|(x or "—")`, список — `join_list()`.
- Разметка непрозрачна: `markup("…") + "…"` — ошибка; строки сначала соединяют, потом вызывают `markup()`.
- Функция из `|code:`, вызванная как команда `|f(…)`, вставляет свой результат; макрос `|def` получает тело вызова параметром `body`.

```
|(None)

|code:
def жирно(текст):
    return markup("|*{" + текст + "}")
|end
|def позначка(текст, body):
|.Note: |*{|(текст):} |(body)
|enddef
|позначка("Увага"):
Текст із |жирно("виділенням").
|end
```

10.11. Чего в ядре нет

Документ должен собираться одинаково на любой машине, поэтому в ядре нет ввода-вывода (кроме `load_*`), часов (`build_date()` фиксируется `--build-date`), случайных чисел и доступа к внутренностям Python. Запрещены `import` (модули подключаются `|import` в разметке), `class` (данные — `record`), `try`/`raise` (ошибка останавливает сборку с позицией; `fail("…")` — намеренная ошибка), `lambda`, `yield`, `async`, `global`, `nonlocal`, `del`, `with`, множества, побитовые операторы, аннотации, `else` у циклов, декораторы кроме `@export`, имена вида `__class__`. Всё это находится ещё при разборе кода, до вычисления; полный список с заменами — в примере (`forbidden.prml`) и в разделе 12.2 спецификации.

```
|code:
квадрат = lambda x: x * x
|end
```

11. Библиотеки: разработка и подключение

Библиотека Primula — каталог с модулями `.m.prml` и их данными, который используют несколько проектов: проверка реквизитов, имена в падежах, графики платежей, QR платежа. Её разрабатывают, проверяют и выпускают отдельно от транслятора и от документов: библиотека — это обычный текст Primula, она не меняет язык и не требует сборки транслятора.

Готовый образец — `examples/11-library`: библиотека `shared/ua-iban` (IBAN UA: проверка, группы по четыре знака, построение) и проект `project/`, который подключает её символической ссылкой.

11.1. Что может библиотека

Модуль содержит только определения: функции и константы в `|code:`, макросы `|def`, `|import` других модулей (раздел «Модули: макросы и функции»). Ему доступно всё ядро — диалект Python (страница «Код: диалект Python») — и функции стандартной библиотеки: числа и деньги, слова и падежи, даты и рабочие дни, `load_csv`, `load_yaml`, `load_json`, изображение из строки SVG (`img(svg=…)`).

Чего у модуля нет, и что делать:

Нет	Вместо этого
кода Python, `import`, ввода-вывода, сети, часов	данные — файлы рядом с модулем (`load_yaml("data/…")`), дата — аргументом функции
`doc.*` документа (модуль вычисляется один раз на сборку)	язык и дату документ передаёт аргументом (`lang="uk"`); функции стандартной библиотеки внутри модуля сами следуют языку документа
`ord()`, `chr()`, побитовых операций	позиция в строке-алфавите: `"АБС…".find(ch)`; таблицы значений
`assert`	`fail("…")` — останавливает сборку с сообщением и местом вызова
тяжёлых вычислений	цикл ядра выполняет порядка десятков тысяч шагов в секунду, лимит — 20 с на вычисление; алгоритм на сотни тысяч шагов (QR-код, хеш) просят как функцию транслятора, а библиотека строит поверх неё

11.2. Устройство каталога

```
ua-iban/                  библиотека (свой репозиторий или каталог)
  README.md               назначение, версия, лицензия, функции, источники правил
  project.p.prml          корень для тестов: tests/ импортирует ../iban
  iban.m.prml             публичный модуль: имя файла — имя пространства (iban.*)
  _tables.m.prml          вспомогательный модуль (необязательно)
  data/                   данные модуля: YAML, CSV, JSON
  tests/iban-test.prml    документы-тесты
  run-tests.sh            primula check для каждого теста
```

- Имена без `_` в начале — интерфейс библиотеки; имена с `_` закрыты (`iban._digits` — ошибка). Меняйте закрытые имена свободно, открытые — только с новой версией.
- Первая строка тела функции — строка-описание на английском, как комментарии кода: `"""IBAN in groups of four."""`. Её показывает исходник, в дальнейшем — подсказки Studio (направление Д3).

- Версия — константа `VERSION = "0.1.0"` в главном модуле и запись в README; несовместимое изменение — новая старшая цифра.
- Файл проекта `project.p.prml` нужен тестам: без него корень — каталог теста, и `|import("../iban")` выходит за корень (P2613). Проект, который подключает библиотеку, этот файл не читает.
- Пути `load_*` и `|import` внутри библиотеки — от файла модуля, без `../` за пределы каталога библиотеки.

11.3. Модуль

```
---
kind: module
---
|code:
VERSION = "0.1.0"
_LETTERS = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"


def normalized(text):
    """IBAN without spaces, in capitals."""
    return "".join(text.split()).upper()


def problem(text):
    """Why the text is not an IBAN UA, or "" if it is valid."""
    s = normalized(text)
    if not s.startswith("UA") or len(s) != 29 or not s[2:].isdigit():
        return "очікується UA і 27 цифр"
    digits = "".join([c if c.isdigit() else str(_LETTERS.find(c) + 10) for c in s[4:]
+ s[:4]])
    return "" if int(digits) % 97 == 1 else "не збігається контрольне число"


def check(text):
    """The normalized IBAN; a wrong one stops the build with the reason."""
    reason = problem(text)
    if reason:
        fail(f"IBAN «{text}»: {reason}")
    return normalized(text)
|end
```

Ошибку данных документа библиотека сообщает через `fail()`: сборка останавливается с сообщением библиотеки, местом `fail` в модуле и строкой документа, откуда пришёл вызов. Для мягкой проверки рядом даётся функция, которая возвращает причину (`problem`) или `True`/`False`.

11.4. Тесты

Тест — обычный документ: код сравнивает результаты с ожидаемыми и вызывает `fail()` при первом расхождении. Вызовы на верхнем уровне `|code:` — действия, они выполняются при каждой проверке.

```
|import("lib/iban")
|code:
def expect(name, got, wanted):
    if got != wanted:
        fail(f"{name}: отримано {repr(got)}, очікувалося {repr(wanted)}")
```

```
expect("check", iban.check("ua21 3223 1300 0002 6007 2335 6600 1"),
"UA213223130000026007233566001")
expect("problem", iban.problem("UA223223130000026007233566001"), "не збігається
контрольне число")
|end
Бібліотека ua-iban |(iban.VERSION).
```

`primula check tests/iban-test.prml` завершается кодом 0, если все ожидания совпали, и кодом 2 с сообщением `fail` иначе; `run-tests.sh` образца проверяет все `tests/*.prml`. Проверяйте граничные случаи и ошибки, а не только удачный путь: неправильная контрольная цифра, неверная длина, чужая страна.

11.5. Подключение к проекту

Три способа, от удобного для разработки к самому воспроизводимому:

Способ	Как	Когда
символическая ссылка	`ln -s ../shared/ua-iban lib` в корне проекта	разработка, несколько проектов на одной машине; сборка выдаёт предупреждение P2639
копия	каталог библиотеки копируется в `lib/` проекта	выпуск документа: проект собирается одинаково без внешних каталогов
подмодуль Git	`git submodule add <url> lib/ua-iban`	проект в Git: версия библиотеки закреплена коммитом

```
cd мій-проєкт
ln -s ../shared/ua-iban lib

|import("lib/iban")
IBAN: |(iban.grouped(рахунок)) (код банку |(iban.bank_code(рахунок))).
```

Ссылка проверяется по записанному пути (решение 136): `lib` внутри проекта может вести куда угодно, а `../` в тексте документа и модуля по-прежнему не выходит за корень. Файлы библиотеки попадают в отчёт сборки и в `--embed-sources` как `lib/iban.m.prml`, с хешами — по отчёту видно, какой версией библиотеки собран документ.

11.6. Данные с датой действия

Ставки, справочники и календари меняются со временем, а старый документ должен собираться так же, как при выпуске. Храните значения с датой начала действия и выбирайте по дате, которую передаёт документ (дата документа, а не дата сборки):

```
vat:
  - {from: 2011-01-01, value: 20}

|code:
_rates = load_yaml("lib/data/rates.yaml")


def vat_on(day):
    """VAT rate on a date; the document passes doc.created or the contract date."""
    found = None
    for r in _rates.vat:
        if r["from"] <= day:
            found = r["value"]
    if found is None:
        fail(f"немає ставки на {fmt_date(day, 'short')}")
    return found
|end
Ставка ПДВ: |(vat_on(date(2026, 10, 26))) %.
```

Новое значение добавляется строкой с новой датой; старые строки не меняются.

11.7. Выпуск

- README: назначение, функции с примерами, версия, лицензия, источники правил (стандарт, закон, постановление) — и дата, на которую они сверены.
- Перед выпуском: `run-tests.sh` без ошибок; номер в `VERSION` и в README совпадает; изменения описаны.
- Не добавляйте в библиотеку реальные персональные данные и реквизиты в примерах — используйте заведомо учебные значения.

Для разработки библиотеки с помощью AI есть skill `skills/primula-library`: он ведёт ту же схему — модуль, тесты, проверка транслятором.

Приложение A. Конструкции

Сгенерировано `tools/author_docs.py appendices` из справочника транслятора; не править вручную.

Команды

Текст

Запись	Назначение	Аргументы	§
`|link`	внешняя гиперссылка; без тела текст — сам адрес	`url`	4.2
`|fmt`	прямое форматирование фрагмента (предупреждение: лучше символьные стили)	`**props`	4.4
`|set`	стили конструкций до конца охватывающего окружения	`**keys`	4.5
`|block`	стиль блока для группы элементов	`style`, `when`	4.3
`|list`	стиль одного списка: строка перед первым элементом или окружение	`style`	5.4

Структура

Запись	Назначение	Аргументы	§
`|appendix`	приложение: обозначение, название, своя область нумерации, новая страница	`id`, `title`, `subtitle`, `number`, `letter`, `page`, `block`, `toc`, `toc_title`, `toc_level`, `when`	5.5
`|scope`	область нумерации с независимыми счётчиками	`name`, `prefix`	5.5
`|toc`	оглавление (указатель, обновляемый Writer)	`max_level`, `title`, `page_break`, `track`	5.9
`|list_of`	перечень рисунков, таблиц, сокращений или алфавитный указатель	`kind`, `title`, `page_break`, `track`	5.9
`|toc_entry`	строка оглавления с закладкой на следующий абзац	`text`, `level`	5.9
`|footnote`	сноска внизу страницы	`id`	5.10
`|endnote`	концевая сноска	`id`	5.10

Таблицы

Запись	Назначение	Аргументы	§
`|table`	таблица: строки |: … |& … или строки из данных	`cols`, `style`, `width`, `align`, `header`, `footer`, `repeat_header`, `keep_rows`, `may_break`, `frame`, `rules`, `caption`, `id`, `block`, `when`, `data`, `fields`, `titles`, `live`	6.2
`|row`	строка таблицы в развёрнутой форме	`align`, `valign`, `padding`, `background`, `paragraph`, `border`, `height`, `min_height`, `keep`, `when`	6.4
`|cell`	ячейка таблицы в развёрнутой форме	`span`, `rows`, `align`, `valign`, `padding`, `background`, `paragraph`, `border`	6.4
`|caption`	подпись таблицы с разметкой		5.8

Страницы и графика

Запись	Назначение	Аргументы	§
`|page`	смена стиля страницы: отсюда или для окружения	`style`, `restart`, `number_format`	7.3
`|newpage`	разрыв страницы		7.4

Запись	Назначение	Аргументы	§
`|newcolumn`	переход к следующей колонке		7.4
`|section`	раздел с колонками	`cols`, `widths`, `gap`, `rule`, `balance`, `id`, `block`, `when`	7.5
`|header`	верхний колонтитул стиля страницы	`page`, `which`	7.6
`|footer`	нижний колонтитул стиля страницы	`page`, `which`	7.6
`|field.page`	номер страницы (поле Writer)		7.6
`|field.pages`	количество страниц (поле Writer)		7.6
`|field.chapter`	название текущего заголовка (поле Writer)	`level`	7.6
`|field.title`	название документа из метаданных (поле Writer)		7.6
`|img`	изображение без подписи	`path`, `key`, `svg`, `place`, `valign`, `x`, `y`, `rel`, `anchor`, `wrap`, `layer`, `gap`, `width`, `height`, `rotate`, `mirror`, `crop`, `opacity`, `border`, `padding`, `style`, `alt`, `when`	8.2
`|figure`	нумерованный рисунок; тело — подпись	`path`, `key`, `svg`, `id`, `block`, `caption`, `place`, `valign`, `x`, `y`, `rel`, `anchor`, `wrap`, `layer`, `gap`, `width`, `height`, `rotate`, `mirror`, `crop`, `opacity`, `border`, `padding`, `style`, `alt`, `when`	8.3
`|frame`	текстовый блок	`id`, `block`, `place`, `valign`, `x`, `y`, `rel`, `anchor`, `wrap`, `layer`, `gap`, `width`, `height`, `rotate`, `mirror`, `crop`, `opacity`, `border`, `padding`, `style`, `alt`, `when`	8.4
`|watermark`	водяной знак на всех или перечисленных стилях страниц	`mark`, `pages`, `angle`, `color`, `size`, `opacity`, `font`	8.6
`|parallel`	параллельные потоки: тексты рядом, сегменты которых начинаются на одном уровне (двуязычный договор, старая и новая редакция, текст и комментарий)	`langs`, `tracks`, `primary`, `widths`, `gap`, `rule`, `header`, `parity`, `style`, `keep_rows`, `id`	7.7
`|segment`	сегмент |parallel: блок на каждый поток, начала на одном уровне	`id`, `span`, `parity`, `keep`, `status`, `when`	7.7
`|track`	блок потока в сегменте; кончается следующим |track или концом сегмента	`track`, `status`, `rev`, `when`	7.7

Техническая документация

Запись	Назначение	Аргументы	§
`|abbr`	сокращение: определение и употребление или употребление	`short`, `full`, `text`, `show`	9.4
`|index`	элемент алфавитного указателя	`term`, `sub`	9.5

Фрагменты, модули, части

Запись	Назначение	Аргументы	§
`|include`	вставить фрагмент	`path`, `part`, `key`, `ns`, `rename`, `level_shift`, `**params`	10.2
`|import`	подключить модуль определений	`path`, `*names`, `alias`	10.3
`|document`	часть сводного документа (своя область, параметры, новая страница)	`path`, `scope`, `params`, `page`, `level_shift`, `when`	10.7
`|part`	именованная часть фрагмента для |include(part=…)	`name`	10.2
`|export`	экспортировать имена фрагмента в документ	`*names`, `**names`	10.4

Код и данные

Запись	Назначение	Аргументы	§
`|let`	определить имена	`**names`, `export`, `override`	10.4
`|code`	блок кода вычислительного ядра		3.7
`|comment`	блочный комментарий		3.7
`|verbatim`	буквальный блок	`end`	3.7
`|if`	условное содержимое	`condition`	9.2
`|elif`	другая ветка |if	`condition`	9.2
`|else`	последняя ветка |if		9.2
`|for`	цикл по списку	`loop`	6.9
`|def`	макрос: пользовательская команда	`signature`	3.2
`|call`	динамический вызов функции как команды	`function`, `*args`	3.2

Краткие формы

Запись	Назначение	§
`||`	литерал |	3.3
`|{ |}`	литеральные фигурные скобки	3.3
`|;`	пустой разделитель: граница имени команды	3.3
`|%`	комментарий до конца строки	3.7
`|(…)`	значение выражения как текст	4.2
`|\`	разрыв строки в абзаце	3.3
`|~`	неразрывный пробел	3.3
`|*{…}`	полужирный	3.3
`|/{…}`	курсив	3.3
`|_{…}`	подчёркивание	3.3
`|^{…}`	верхний индекс	3.3
`|,{…}`	нижний индекс	3.3
`|–{…}`	зачёркивание	3.3
|{…}`	моноширинный: обозначения, коды	3.3
`|"{…}`	кавычки языка документа	4.2
`|?{…}`	заполнитель черновика	4.2
`|.Им'я{…}`	символьный стиль	4.3
`|.Им'я:`	абзацный стиль (строка или окружение |end.Им'я)	4.3
`|@("id")`	перекрёстная ссылка: номер, label, title, page	5.7
`|<`	заголовок; количество < — уровень (1–6) или level=	5.2
`|>`	пункт; количество > — уровень (окружение |end>)	5.3
`|-`	маркированный элемент списка; длина — уровень	5.4
`|#`	нумерованный элемент списка; длина — уровень	5.4
`|!("kind")`	предупреждение или примечание: danger, warning, caution, notice, note, tip	9.1
`|:`	строка таблицы	6.4
`|&`	ячейка таблицы	6.4
`|=(…)`	отложенная формула ячейки	6.8

Маркеры закрытия

Запись	Назначение	§
`|end`	закрывает ближайшее окружение без проверки	3.5
`|end name`	закрывает именованное окружение с проверкой	3.5
`|end>`	закрывает пункт этого уровня	3.5
`|end!`	закрывает предупреждение	3.5
`|end.Ім'я`	закрывает абзацный стиль	3.5
`|endif`	закрывает |if	3.5
`|endfor`	закрывает |for	3.5
`|enddef`	закрывает |def	3.5

Приложение Б. Функции выражений

Сгенерировано `tools/author_docs.py appendices` из справочника транслятора; не править вручную.

Числа

Вызов	Назначение	Пример → результат	§
`num(x, digits=None, lang)`	число по правилам языка: группы разрядов, запятая	`num(12500, digits=2) → "12 500,00"`	библ. 2
`percent(x, digits=None, lang)`	процент с неразрывным пробелом: «20 %»	`percent(20) → "20 %"`	библ. 2
`round_to(x, step)`	округлить до шага, половина от нуля	`round_to(1234, 50) → 1250`	библ. 2
`ceil_to(x, step)`	округлить до шага вверх	`ceil_to(1201, 100) → 1300`	библ. 2
`floor_to(x, step)`	округлить до шага вниз	`floor_to(1299, 100) → 1200`	библ. 2
`round_money(x)`	округлить до сотых, половина от нуля	`round_money(dec("10.005")) → 10.01`	библ. 2
`round(x, ndigits=None)`	округлить, половина от нуля; без ndigits — целое	`round(2.675, 2) → 2.68`	12.4
`abs(x)`	модуль числа	`abs(-5) → 5`	12.10
`min(*values, key, default)`	наименьшее из аргументов или списка	`min(3, 1, 2) → 1`	12.10
`max(*values, key, default)`	наибольшее из аргументов или списка	`max([3, 1, 2]) → 3`	12.10
`sum(seq, start=0)`	сумма списка	`sum([1.50, 2.25]) → 3.75`	12.10
`dec(x)`	в decimal (из float — кратчайшая десятичная запись)	`dec(0.1) → 0.1`	12.4
`float(x)`	в float; datetime даёт серийное число	`float("2.5") → 2.5`	12.4
`int(x)`	отбросить дробную часть (decimal с масштабом 0)	`int(7.9) → 7`	12.4

Деньги и НДС

Вызов	Назначение	Пример → результат	§
`money(x, code, currency_style="short", lang)`	сумма с валютой: «12 500,00 грн»	`money(12500, "UAH") → "12 500,00 грн"`	библ. 3
`money_words(x, code, kop="digits", cap=False, lang)`	сумма прописью	`money_words(12500, "UAH") → "двенадцать тысяч пятьсот гривен 00 копеек"`	библ. 3
`money_full(x, code, kop="digits", cap=False, currency_style="short", lang)`	сумма цифрами и прописью в скобках (договорная форма)	`money_full(12500.50, "UAH") → "12 500,50 грн (двенадцать тысяч пятьсот гривен 50 копеек)"`	библ. 3
`vat(x, rate)`	налог с суммы без налога	`vat(1000, 20) → 200.00`	библ. 3
`with_vat(x, rate)`	сумма с налогом	`with_vat(1000, 20) → 1200.00`	библ. 3
`vat_in(x, rate)`	налог, содержащийся в сумме с налогом	`vat_in(1200, 20) → 200.00`	библ. 3

Вызов	Назначение	Пример → результат	§
`without_vat(x, rate)`	сумма без налога из суммы с налогом	`without_vat(1200, 20) → 1000.00`	библ. 3
`split(total, shares)`	разделить сумму по долям; части дают сумму	`split(100, [1, 1, 1]) → [33.33, 33.33, 33.34]`	библ. 3

Прописью

Вызов	Назначение	Пример → результат	§
`words(x, code=None, gender="m", case="nom", lang)`	число прописью; words(x, "UAH") — сумма	`words(21, gender="f") → "двадцать одна"`	библ. 4
`ordinal(n, gender="m", case="nom", lang)`	порядковое числительное прописью	`ordinal(21, case="gen") → "двадцать первого"`	библ. 4
`date_words(dt, lang)`	дата прописью	`date_words(date(2026, 10, 12)) → "двенадцатого октября две тысячи двадцать шестого года"`	библ. 4

Даты и время

Вызов	Назначение	Пример → результат	§
`date(y, m, d)`	дата: date(y, m, d) или date("YYYY-MM-DD")	`date(2026, 10, 12) → date("2026-10-12")`	12.3
`datetime(y, m, d, h=0, mi=0, s=0)`	момент: datetime(y, m, d, h, mi, s), строка ISO или серийное число	`datetime("2026-10-12T14:30") → datetime("2026-10-12T14:30:00")`	12.3
`build_date()`	дата начала сборки (--build-date, build_date: проекта)	`build_date() → date("2026-10-12")`	12.7
`weeks(n)`	число суток в n неделях: dt + weeks(2)	`weeks(2) → 14`	библ. 5
`hours(n)`	число суток в n часах: dt + hours(36)	`hours(36) → 1.5`	библ. 5
`minutes(n)`	число суток в n минутах	`date(2026, 10, 12) + minutes(90) → datetime("2026-10-12T01:30:00")`	библ. 5
`add_months(dt, n)`	сдвиг на месяцы (31 → последний день месяца)	`add_months(date(2026, 1, 31), 1) → date("2026-02-28")`	библ. 5
`add_years(dt, n)`	сдвиг на годы	`add_years(date(2028, 2, 29), 1) → date("2029-02-28")`	библ. 5
`fmt_date(dt, format="short", lang)`	дата текстом: short, long, contract или шаблон (ДСТУ 4163:2020)	`fmt_date(date(2026, 10, 12), "contract") → "«12» октября 2026 г."`	библ. 5
`fmt_time(dt, format="short", lang)`	время текстом: short или шаблон {hh}:{mi}	`fmt_time(datetime(2026, 10, 12, 14, 30, 0)) → "14:30"`	библ. 5
`fmt_datetime(dt, format="short", lang)`	дата и время текстом	`fmt_datetime(datetime(2026, 10, 12, 14, 30, 0), "long") → "12 октября 2026 года, 14:30"`	библ. 5
`weekday(dt, case="nom", lang)`	название дня недели (падеж nom или acc)	`weekday(date(2026, 10, 12)) → "понедельник"`	библ. 5
`month_name(dt, case="nom", lang)`	название месяца (падеж nom, gen, loc)	`month_name(date(2026, 10, 12), case="gen") → "октября"`	библ. 5
`is_workday(dt, calendar)`	рабочий ли день (по календарю проекта)	`is_workday(date(2026, 10, 10)) → False`	библ. 5
`workdays(dt, n, calendar)`	дата через n рабочих дней после dt (n < 0 — до)	`workdays(date(2026, 10, 9), 1) → date("2026-10-12")`	библ. 5

Единицы и длины

Вызов	Назначение	Пример → результат	§
`qty(x, unit, tol, digits=None, lang)`	значение с единицей: «230 В», tol= — допуск	`qty(230, "V") → "230 В"`	библ. 6
`qty_range(low, high, unit, digits=None, lang)`	диапазон с единицей: «100–240 В»	`qty_range(-20, 55, "degC") → "−20…+55 °C"`	библ. 6
`convert(x, from, to)`	пересчитать между единицами одной величины	`convert(1.5, "kW", "W") → 1500`	библ. 6
`length(text)`	длина из строки: length("10mm")	`length("10mm") + 5*mm → length("15mm")`	12.3
`mm`	константа длины: 10*mm	`10*mm → length("10mm")`	12.3
`cm`	константа длины: 10*cm	`10*cm → length("10cm")`	12.3
`pt`	константа длины: 10*pt	`10*pt → length("10pt")`	12.3
`inch`	константа длины: 10*inch	`10*inch → length("10inch")`	12.3

Текст

Вызов	Назначение	Пример → результат	§
`plural(n, *forms, frac, lang)`	форма слова по числу: plural(n, "день", "дня", "дней")	`plural(5, "день", "дні", "днів", lang="uk") → "днів"`	библ. 7
`join_list(items, last, sep=", ", lang)`	соединить список: «А, Б и В»	`join_list(["А", "Б", "В"], last="та") → "А, Б та В"`	библ. 7
`cap(s)`	заглавная первая буква	`cap("київ") → "Київ"`	библ. 7
`translit(s, lang)`	транслитерация (uk — постановление КМУ № 55, ru — ICAO Doc 9303)	`translit("Згорани", lang="uk") → "Zghorany"`	библ. 7
`sort_key(s, lang)`	ключ сортировки по алфавиту языка: sorted(names, key=sort_key)	`sorted(["Ґава", "Гора", "Єва", "Есе"], key=sort_key) → ['Гора', 'Есе', 'Єва', 'Ґава']`	библ. 7
`nbsp_short(s, lang)`	неразрывный пробел после коротких предлогов и союзов	`nbsp_short("в домі та у дворі", lang="uk") → "в домі та у дворі"`	библ. 7
`str(x)`	значение строкой (даты — ISO, числа — с точкой)	`str(2.20) → "2.20"`	12.10
`repr(x)`	значение в виде кода	`repr("a") → "'a'"`	12.10

Списки и записи

Вызов	Назначение	Пример → результат	§
`len(seq)`	длина: символы, элементы, ключи	`len("Київ") → 4`	12.10
`range(start, stop, step=1)`	числа от start до stop (не включительно) с шагом	`range(1, 10, 3) → [1, 4, 7]`	12.10
`enumerate(seq, start=0)`	пары (номер, элемент)	`enumerate(["a", "b"], start=1) → [(1, 'a'), (2, 'b')]`	12.10
`zip(*lists)`	кортежи элементов с одинаковым индексом	`zip([1, 2], ["a", "b"]) → [(1, 'a'), (2, 'b')]`	12.10
`sorted(seq, key, reverse=False)`	отсортированный список	`sorted([3, 1, 2]) → [1, 2, 3]`	12.10
`reversed(seq)`	список в обратном порядке	`reversed([1, 2, 3]) → [3, 2, 1]`	12.10
`any(seq)`	есть ли хоть один истинный элемент	`any([0, 1]) → True`	12.10

Вызов	Назначение	Пример → результат	§
`all(seq)`	все ли элементы истинны	`all([1, 2]) → True`	12.10
`list(seq)`	список (из последовательности)	`list("abc") → ['a', 'b', 'c']`	12.10
`tuple(seq)`	кортеж	`tuple([1, 2]) → (1, 2)`	12.10
`dict(source=None, **items)`	словарь: из пар, записи или key=value	`dict(a=1, b=2) → {'a': 1, 'b': 2}`	12.10
`record(source=None, **fields)`	запись с доступом через точку: r.name	`record(name="Кабель", qty=3) → record(name='Кабель', qty=3)`	12.6
`bool(x)`	логическое значение	`bool("") → False`	12.10
`getattr(r, name, default)`	поле записи по имени, со значением по умолчанию	`getattr(record(a=1), "b", 0) → 0`	12.6
`hasattr(r, name)`	есть ли у записи поле	`hasattr(record(a=1), "a") → True`	12.6
`type(x)`	имя типа: "decimal", "str", "datetime"…	`type(1.5e3) → "float"`	12.3

Данные

Вызов	Назначение	Пример → результат	§
`group_by(items, key)`	словарь «значение ключа → список записей»	`group_by([record(k="a", v=1), record(k="a", v=2)], "k") → {'a': [record(k='a', v=1), record(k='a', v=2)]}`	библ. 8
`sum_by(items, key, value)`	суммы поля по группам	`sum_by([record(k="a", v=1), record(k="a", v=2)], "k", "v") → {'a': 3}`	библ. 8
`distinct(items, key)`	уникальные значения поля	`distinct([record(k="a"), record(k="b"), record(k="a")], "k") → ['a', 'b']`	библ. 8
`load_csv(path, types, sep)`	строки файла CSV как записи	`load_csv("data/items.csv")`	10.5
`load_yaml(path)`	данные файла YAML (записи и списки)	`load_yaml("data/order.yaml")`	10.5
`load_json(path)`	данные файла JSON	`load_json("data/order.json")`	10.5
`load_text(path)`	весь текстовый файл как строка (UTF-8; например, SVG для |img(svg=…))	`load_text("img/sign.svg")`	10.5
`load_ods(path, sheet, range, header=True, types)`	строки листа файла ODS как записи (сохранённые результаты формул)	`load_ods("data/items.ods", sheet="Позиції")`	10.5
`load_xlsx(path, sheet, range, header=True, types)`	строки листа файла XLSX как записи (сохранённые результаты формул)	`load_xlsx("data/items.xlsx", sheet="Позиції")`	10.5

Математика (std/math)

Вызов	Назначение	Пример → результат	§
`math.sqrt(x)`	квадратный корень; в документе: |import("std/math")	`math.sqrt(2) → 1.4142135623730951`	библ. 9
`math.exp(x)`	e в степени x; в документе: |import("std/math")	`math.exp(1) → 2.718281828459045`	библ. 9
`math.log(x)`	натуральный логарифм; в документе: |import("std/math")	`math.log(math.e) → 1.0`	библ. 9
`math.log10(x)`	десятичный логарифм; в документе: |import("std/math")	`math.log10(1000) → 3.0`	библ. 9
`math.sin(x)`	синус (радианы); в документе: |import("std/math")	`math.sin(math.pi / 2) → 1.0`	библ. 9
`math.cos(x)`	косинус (радианы); в документе: |import("std/math")	`math.cos(0) → 1.0`	библ. 9

Вызов	Назначение	Пример → результат	§
`math.tan(x)`	тангенс (радианы); в документе: |import("std/math")	`math.tan(0) → 0.0`	библ. 9
`math.atan2(y, x)`	угол точки (x, y), радианы; в документе: |import("std/math")	`math.atan2(1, 1) → 0.7853981633974483`	библ. 9
`math.floor(x)`	округлить вниз до целого (decimal); в документе: | import("std/math")	`math.floor(-2.5) → -3`	библ. 9
`math.ceil(x)`	округлить вверх до целого (decimal); в документе: | import("std/math")	`math.ceil(2.1) → 3`	библ. 9
`math.pi`	число π (float); в документе: |import("std/math")	`math.pi → 3.141592653589793`	библ. 9
`math.e`	число e (float); в документе: |import("std/math")	`math.e → 2.718281828459045`	библ. 9

QR-коды и кодировки

Вызов	Назначение	Пример → результат	§
`qr_matrix(data, ecc="M", min_version=1, max_version=40, encoding="utf-8")`	QR-код текста (байтовый режим): {version, ecc, mask, size, rows}	`qr_matrix("https://primula.example").version → 2`	библ. 10
`qr_svg(data, ecc="M", min_version=1, max_version=40, encoding="utf-8", size="30mm", quiet=4, circle=0)`	QR-код как SVG для |img(svg=…); circle= оставляет белый круг в центре	`len(qr_svg("https://primula.example")) > 0 → True`	библ. 10
`base64url(text, encoding="utf-8")`	Base64URL байтов текста, без дополнения =	`base64url("Київ") → "0JrQuNGX0LI"`	библ. 10

Проверки и документ

Вызов	Назначение	Пример → результат	§
`fail(*message)`	остановить сборку с ошибкой в этом месте	`fail("no price")`	12.9
`debug(*values)`	записать значения в отчёт сборки, а не в документ	`debug(total)`	12.9
`has(feature)`	есть ли у варианта изделия функция (keys: проекта, --feature)	`has("wifi")`	9.2
`markup(text)`	разметка Primula из строки (content)	`markup("|*{" + name + "}")`	4.2
`img(path, svg, **options)`	значение-изображение для watermark=, | img(значение), |figure(значение); путь или svg=	`img("assets/stamp.png")`	8.6
`col(width, id, type, **props)`	столбец таблицы: col(width, id=…, type=…, align=…)	`col("20mm..35mm", id="sum", type="money", align="right")`	6.3
`track(id, lang, title, **props)`	поток |parallel: track(id, lang=…, title=…, width=…, primary=…)	`track("note", title="Коментар", width="28%", numbered=False, optional=True)`	7.7

Формулы таблиц

Вызов	Назначение	Пример → результат	§
`this_row(c)`	значение в столбце с текущей строки	`this_row("qty") * this_row("price")`	6.8
`cell(r, c)`	значение ячейки	`cell(2, "sum")`	6.8
`column(c, rows)`	список значений столбца (строки тела или rows=)	`sum(column("sum"))`	6.8
`body_rows()`	номера строк между заголовком и итогами	`body_rows()`	6.8
`avg(seq)`	среднее значение списка	`avg(column("qty"))`	6.8

Вызов	Назначение	Пример → результат	§
`count(seq)`	количество непустых значений	`count(column("qty"))`	6.8

Код стилей

Вызов	Назначение	Пример → результат	§
`paragraph_style(name, parent, **props)`	определить абзацный стиль: имя, parent=…, свойства (11.3)	`paragraph_style("MyStyle", parent="…")`	11.2
`character_style(name, parent, **props)`	определить символьный стиль: имя, parent=…, свойства (11.3)	`character_style("MyStyle", parent="…")`	11.2
`block_style(name, parent, **props)`	определить блочный стиль: имя, parent=…, свойства (11.3)	`block_style("MyStyle", parent="…")`	11.2
`table_style(name, parent, **props)`	определить табличный стиль: имя, parent=…, свойства (11.3)	`table_style("MyStyle", parent="…")`	11.2
`list_style(name, parent, **props)`	определить списочный стиль: имя, parent=…, свойства (11.3)	`list_style("MyStyle", parent="…")`	11.2
`frame_style(name, parent, **props)`	определить фреймовый стиль: имя, parent=…, свойства (11.3)	`frame_style("MyStyle", parent="…")`	11.2
`page_style(name, parent, **props)`	определить страничный стиль: имя, parent=…, свойства (11.3)	`page_style("MyStyle", parent="…")`	11.2
`derive(base, name, **props)`	производный стиль: его детерминированное имя для |.(…)	`derive("Clause", left="10mm")`	11.4
`structure_styles()`	стили конструкций структуры: paragraph=, heading=, clause=…	`structure_styles(paragraph="Body")`	11.5
`structure_profile()`	профиль структуры: "business" или "report"	`structure_profile("business")`	5.1
`numbering(headings, heading_format, clause_format, figures, tables, ref_format)`	нумерация заголовков, пунктов, рисунков и таблиц	`numbering(figures="by_chapter")`	5.2
`captions(lang, figure, table)`	слова подписей рисунков и таблиц	`captions("uk", figure="Рисунок", table="Таблиця")`	5.8
`appendices(lang, designation, numbering, letters, page_break, toc, title_format)`	обозначение и оформление приложений	`appendices("uk", designation="ДОДАТОК")`	5.9
`ref_labels(lang)`	слова ссылок |@(id, "label")	`ref_labels("uk", clause="п.")`	5.7
`footnotes(numbering, restart, format, separator)`	нумерация и разделитель сносок	`footnotes(restart="page")`	5.10
`abbreviations(first, sort)`	первое употребление и сортировка сокращений	`abbreviations(first="expand")`	9.4
`typography(percent_space, nbsp_short, nbsp_words)`	типографика: percent_space, nbsp_short, nbsp_words	`typography(nbsp_short=True)`	библ. 7
`admonition_symbols(danger, warning, caution, notice, note, tip)`	изображения знаков безопасности предупреждений	`admonition_symbols(danger="assets/signs/w012.svg")`	9.1
`admonition_words(lang, danger, warning, caution, notice, note, tip, notes)`	сигнальные слова предупреждений	`admonition_words("uk", danger="НЕБЕЗПЕКА")`	9.1
`review_marks(lang, true, false, watermark)`	тексты пометок и водяного знака --review	`review_marks("uk", watermark="НЕ ДЛЯ ВИПУСКУ")`	9.2
`watermark()`	водяной знак на стилях страниц	`watermark("ПРОЄКТ")`	8.6
`header()`	верхний колонтитул стиля страницы	`header("Portrait", markup("…"))`	7.6

Вызов	Назначение	Пример → результат	§
`footer()`	нижний колонтитул стиля страницы	`footer("Portrait", markup("…"))`	7.6
`placeholders()`	заполнители |?{…}: "mark" (черновик) или "final"	`placeholders("final")`	4.2
`quotes()`	кавычки языка для |"{…}	`quotes("de", ("„", "“"), (",", "‘"))`	4.2
`currency(code, lang, short, units, frac, gender, minor, minor_gender, minor_digits)`	добавить или изменить валюту библиотеки	`currency("PLN", "uk", short="zł")`	библ. 1
`unit(unit, lang, symbol, quantity, factor, offset)`	добавить или изменить единицу библиотеки	`unit("rpm", "uk", symbol="об/хв", quantity="frequency", factor=1/60)`	библ. 1
`month_names(lang, nom, gen, loc)`	названия месяцев языка	`month_names("uk", gen=["січня", …])`	библ. 1

Проект

Вызов	Назначение	Пример → результат	§
`build(source, output, params, keys, pdf)`	зарегистрировать цель сборки проекта	`build("act.prml", params=row, output=f"out/act-{row.no}.odt")`	10.7
`batch_mode()`	True во время сборки всего проекта	`batch_mode()`	10.6

Методы

Вызов	Назначение	Пример → результат	§
`str.split(sep=None, maxsplit=-1)`	разбить по разделителю (без него — по пробелам)	`"a, b".split(", ") → ['a', 'b']`	12.10
`str.rsplit(sep=None, maxsplit=-1)`	разбить справа	`"a.b.c".rsplit(".", 1) → ['a.b', 'c']`	12.10
`str.splitlines(keepends=False)`	список строк	`"a\nb".splitlines() → ['a', 'b']`	12.10
`str.join(items)`	соединить строки этой строкой	`", ".join(["a", "b"]) → "a, b"`	12.10
`str.strip(chars=None)`	без пробелов (или символов) с обоих концов	`"  a ".strip() → "a"`	12.10
`str.lstrip(chars=None)`	без пробелов в начале	`"  a".lstrip() → "a"`	12.10
`str.rstrip(chars=None)`	без пробелов в конце	`"a  ".rstrip() → "a"`	12.10
`str.replace(old, new, count=-1)`	заменить подстроку	`"a-b".replace("-", "/") → "a/b"`	12.10
`str.startswith(prefix)`	начинается ли с префикса	`"ДСТУ 3008".startswith("ДСТУ") → True`	12.10
`str.endswith(suffix)`	заканчивается ли суффиксом	`"act.prml".endswith(".prml") → True`	12.10
`str.find(sub, start=None, end=None)`	индекс подстроки или −1	`"abc".find("c") → 2`	12.10
`str.rfind(sub, start=None, end=None)`	индекс последнего вхождения или −1	`"abca".rfind("a") → 3`	12.10
`str.count(sub, start=None, end=None)`	количество вхождений	`"banana".count("a") → 3`	12.10
`str.upper()`	заглавными буквами	`"київ".upper() → "КИЇВ"`	12.10
`str.lower()`	строчными буквами	`"КИЇВ".lower() → "київ"`	12.10

Вызов	Назначение	Пример → результат	§
`str.capitalize()`	первая заглавная, остальные строчные	`"кИЇВ".capitalize() → "Київ"`	12.10
`str.title()`	каждое слово с заглавной	`"нова пошта".title() → "Нова Пошта"`	12.10
`str.isdigit()`	только ли цифры	`"2026".isdigit() → True`	12.10
`str.isalpha()`	только ли буквы	`"Київ".isalpha() → True`	12.10
`str.isspace()`	только ли пробелы	`" ".isspace() → True`	12.10
`str.zfill(width)`	дополнить нулями слева	`"7".zfill(3) → "007"`	12.10
`str.format(*args, **kwargs)`	подставить значения: "{0} — {name}"	`"{0} шт.".format(3) → "3 шт."`	12.10
`list.append(x)`	добавить элемент (внутри функций)	`[1].append(2)`	12.10
`list.extend(seq)`	добавить элементы списка	`[1].extend([2, 3])`	12.10
`list.insert(i, x)`	вставить элемент по индексу	`[1].insert(0, 0)`	12.10
`list.pop(i=-1)`	удалить и вернуть элемент (последний)	`[1, 2].pop() → 2`	12.10
`list.remove(x)`	удалить первый равный элемент	`[1, 2].remove(1)`	12.10
`list.index(x)`	индекс элемента	`["a", "b"].index("b") → 1`	12.10
`list.count(x)`	количество равных элементов	`[1, 1, 2].count(1) → 2`	12.10
`list.sort(key, reverse=False)`	отсортировать на месте	`[2, 1].sort()`	12.10
`list.reverse()`	обратить на месте	`[1, 2].reverse()`	12.10
`dict.get(key, default=None)`	значение по ключу или по умолчанию	`{"a": 1}.get("b", 0) → 0`	12.10
`dict.keys()`	список ключей	`{"a": 1}.keys() → ['a']`	12.10
`dict.values()`	список значений	`{"a": 1}.values() → [1]`	12.10
`dict.items()`	пары (ключ, значение)	`{"a": 1}.items() → [('a', 1)]`	12.10
`dict.update(other)`	добавить пары другого словаря	`{"a": 1}.update({"b": 2})`	12.10
`dict.pop(key, default)`	удалить ключ и вернуть значение	`{"a": 1}.pop("a") → 1`	12.10
`dict.setdefault(key, default=None)`	значение по ключу; отсутствует — установить по умолчанию	`{}.setdefault("a", 1) → 1`	12.10

Приложение B. Ключи YAML-заголовка

Сгенерировано `tools/author_docs.py appendices` из справочника транслятора; не править вручную.

Ключ	Файлы	Тип	Назначение	Пример → результат	§
`kind`	document, fragment, module, styles, project	string	вид файла; следует из подрасширения (.f, .m, .s, .p), в проекте обязателен	`kind: document`	2.2
`primula`	document, fragment, module, styles, project	mapping	объявление языка: {version: "1.1"}, при необходимости версия библиотеки	`primula: {version: "1.1"}`	2.3
`primula.version`	document, fragment, module, styles, project	string	версия языка; принимается только 1.1	`version: "1.1"`	2.3
`primula.stdlib`	document, fragment, module, styles, project	string	минимальная версия стандартной библиотеки	`stdlib: "1.1"`	13
`doc`	document	mapping	метаданные ODT документа; в тексте |(doc.title)	`doc:`	2.3
`doc_defaults`	project	mapping	значения по умолчанию метаданных (doc:) всех документов проекта; документ переопределяет их поштучно	`doc_defaults:`	10.6
`doc.title`	document	string	название (dc:title) (рекомендуемый)	`title: "Договір поставки"`	2.3
`doc_defaults.title`	project	string	название (dc:title)	`title: "Договір поставки"`	10.6
`doc.subject`	document	string	тема (dc:subject)	`subject: "Поставка обладнання"`	2.3
`doc_defaults.subject`	project	string	тема (dc:subject)	`subject: "Поставка обладнання"`	10.6
`doc.description`	document	string	описание (dc:description)	`description: "Типова форма"`	2.3
`doc_defaults.description`	project	string	описание (dc:description)	`description: "Типова форма"`	10.6
`doc.creator`	document	string	автор (meta:initial-creator, dc:creator) (рекомендуемый)	`creator: "Іван Петренко"`	2.3
`doc_defaults.creator`	project	string	автор (meta:initial-creator, dc:creator) (рекомендуемый)	`creator: "Іван Петренко"`	10.6
`doc.organization`	document	string	организация (пользовательское свойство)	`organization: "ТОВ «Постачальник»"`	2.3

Ключ	Файлы	Тип	Назначение	Пример → результат	§
`doc_defaults.organization`	project	string	организация (пользовательское свойство) (рекомендуемый)	`organization: "ТОВ «Постачальник»"`	10.6
`doc.language`	document	language	язык документа (dc:language); иначе язык проекта (рекомендуемый)	`language: uk-UA`	2.3
`doc_defaults.language`	project	language	язык документа (dc:language); иначе язык проекта	`language: uk-UA`	10.6
`doc.identifier`	document	string	номер документа (пользовательское свойство)	`identifier: "Д-2026-017"`	2.3
`doc_defaults.identifier`	project	string	номер документа (пользовательское свойство)	`identifier: "Д-2026-017"`	10.6
`doc.revision`	document	string	редакция (пользовательское свойство)	`revision: "2"`	2.3
`doc_defaults.revision`	project	string	редакция (пользовательское свойство)	`revision: "2"`	10.6
`doc.status`	document	string	статус (пользовательское свойство)	`status: "проєкт"`	2.3
`doc_defaults.status`	project	string	статус (пользовательское свойство)	`status: "проєкт"`	10.6
`doc.created`	document	date	дата создания (meta:creation-date); иначе дата сборки (рекомендуемый)	`created: 2026-10-12`	2.3
`doc_defaults.created`	project	date	дата создания (meta:creation-date); иначе дата сборки	`created: 2026-10-12`	10.6
`doc.modified`	document	date	дата изменения (dc:date); иначе дата сборки	`modified: 2026-10-16`	2.3
`doc_defaults.modified`	project	date	дата изменения (dc:date); иначе дата сборки	`modified: 2026-10-16`	10.6
`doc.keywords`	document	strings	ключевые слова (meta:keyword): строка или список	`keywords: [договір, поставка]`	2.3
`doc_defaults.keywords`	project	strings	ключевые слова (meta:keyword): строка или список	`keywords: [договір, поставка]`	10.6
`params`	document, fragment, project	mapping	параметры со значениями по умолчанию; цель или --param их переопределяет	`params: {vat: true, delivery_days: 5}`	10.6
`structure`	document	mapping	настройки профиля структуры документа	`structure: {numbering: {headings: false}}`	5.1
`styles`	document, project	paths	модули стилей: список путей или один путь (std/styles/default — встроенный)	`styles: [styles/business.s.prml]`	11.7
`references`	document	list	документы проекта, на цели которых ссылается документ (имена из ref_documents проекта)	`references: [contract]`	5.7
`defaults`	styles	mapping	свойства по умолчанию: шрифт, размер, язык	`defaults: {font: Roboto, size: 11pt, language: uk-UA}`	11.1
`paragraphs`	styles	mapping	стили абзацев	`paragraphs:`	11.2
`characters`	styles	mapping	стили символов	`characters:`	11.2
`blocks`	styles	mapping	стили блоков	`blocks:`	11.2
`tables`	styles	mapping	стили таблиц	`tables:`	11.2

Ключ	Файлы	Тип	Назначение	Пример → результат	§
`lists`	styles	mapping	стили списков	`lists:`	11.2
`frames`	styles	mapping	стили врезок	`frames:`	11.2
`extends`	styles	path	базовый модуль стилей, подключается слоем перед этим (решение 133)	`extends: std/styles/business`	11.7
`pages`	styles	mapping	стили страниц	`pages:`	11.2
`name`	project	string	название проекта	`name: "Договори 2026"`	10.6
`language`	project	language	язык документов по умолчанию	`language: uk-UA`	10.6
`extends`	project	path	базовый проект: его параметры, ключи, документы для ссылок, стили и метаданные по умолчанию наследуются	`extends: ../../project.p.prml`	10.6
`on_conflict`	project	string	конфликты экспорта фрагментов: error, first, last	`on_conflict: error`	10.4
`keys`	project	mapping	карта ключей: имя → путь	`keys: {logo: assets/logo.svg}`	9.3
`limits`	project	mapping	лимиты ядра: time, recursion, size	`limits: {time: 20, recursion: 500, size: 10_000_000}`	12.8
`limits.time`	project	number	лимит времени, секунды	`time: 20`	12.8
`limits.recursion`	project	int	лимит глубины рекурсии	`recursion: 500`	12.8
`limits.size`	project	int	лимит размера значений	`size: 10_000_000`	12.8
`build_date`	project	date	фиксированная дата сборки (today() и даты по умолчанию)	`build_date: 2026-10-12`	12.7
`targets`	project	list	цели сборки primula build project.p.prml	`targets:`	10.7
`targets[].source`	project	path	исходный документ	`- source: contract.prml`	10.7
`targets[].output`	project	path	выходной ODT	`output: out/contract.odt`	10.7
`targets[].params`	project	mapping	значения параметров цели	`params: {vat: false}`	10.7
`targets[].keys`	project	mapping	ключи цели	`keys: {logo: assets/other.svg}`	10.7
`targets[].pdf`	project	any	также PDF: true или его путь	`pdf: true`	10.7
`ref_documents`	project	mapping	документы, на которые ссылаются другие (references: документа): имя: путь или {source, registry}	`ref_documents: {contract: contract.prml}`	10.6

Приложение Г. Сообщения транслятора

Сгенерировано `tools/author_docs.py appendices` из справочника транслятора; не править вручную.

Код	Вид	Сообщение	§
P0001	error	'{command}' не реализовано на этапе прототипа {stage}; результат не записывается	— (временный, для команд прототипа)
P0002	error	внутренняя ошибка транслятора: {detail}	14.2
P0004	error	превышен лимит времени сборки (--timeout)	12.8, 14.1 (--timeout)
P0005	error	не хватает памяти: вычисление превысило лимит памяти процесса транслятора	12.8
P0006	error	прервано: вычисление ввода остановлено	14.1 (primula console, решение 128)
P0101	error	не удаётся прочитать файл '{path}': {reason}	14.1
P0102	error	файл не в кодировке UTF-8: недопустимый байт в позиции {byte_offset}	2.1
P0103	error	не удаётся записать файл '{path}': {reason}	14.3
P0105	error	--overlay '{text}' нужно записывать как ФАЙЛ=БУФЕР, и файл буфера должен существовать	14.1, решение 124
P0106	error	primula parallel: {problem}{value}	7.7, 14.1, решение 134
P0107	error	сегмент {segment}, поток '{track}' не в состоянии '{expected}': {reason} ({status})	7.7, 14.1, решение 134
P1101	error	заголовок YAML не закрыт: нет строки '---'	2.3, A Header
P1102	error	ошибка YAML в заголовке: {problem}	2.3
P1103	error	заголовок YAML должен быть отображением «ключ: значение», а найдено {found}	2.3
P1104	warning	неизвестный ключ '{key}' в заголовке файла вида '{kind}'	2.3, 10.6, 11.1
P1105	error	ключ заголовка '{key}': ожидается {expected}, а найдено {found}	2.3, 10.6, 11.1
P1106	error	неизвестный вид файла '{value}'; ожидается один из: {expected}	2.2
P1107	error	неподдерживаемая версия языка '{value}'; транслятор реализует версию {supported}	0.1, 2.3
P1109	error	несуществующая дата '{value}'	12.6 (решение 25)
P1110	error	повторный ключ '{key}'	YAML 1.2 (уникальность ключей)
P1111	error	неподдерживаемый тег YAML '{tag}'	12.6 (схема core)
P1112	error	рекурсивный псевдоним YAML: значение содержит само себя	12.6
P1113	error	ключ отображения должен быть скалярным значением	12.6
P1114	warning	'{value}' в YAML 1.2 — строка, а не логическое значение	12.6 (решение 25)
P1115	warning	'{value}' читается как целое число {number}: начальные нули теряются	12.6 (решение 25)
P1116	error	ключ 'lines' удалён: режим строк удалён в версии 1.1, строчная форма заканчивается в конце строки	2.3, 3.4, решение 107
P1117	error	недопустимая версия стандартной библиотеки '{value}': ожидается MAJOR.MINOR, например "1.1"	2.3, 13

Код	Вид	Сообщение	§
P1118	error	файл требует стандартную библиотеку {value}; у транслятора {available}	2.3, 13
P1119	error	часовые пояса не поддерживаются: '{value}'	2.3, 12.6, решение 109
P1120	error	ключ заголовка '{key}' переименован: пишите '{new}:' (решение 129)	2.3, 10.6, решение 129
P1201	error	после '|' должно быть имя команды, '(' или краткая форма; найдено {found}	3.1
P1202	error	неизвестная краткая форма '|{char}'	3.1, B.1
P1203	error	'|{char}' не является краткой формой	3.1
P1204	error	'{bracket}' не закрыта	3.2, решение 22
P1205	error	закрывающая '{found}' не соответствует открывающей скобке; ожидается '{expected}'	3.2
P1206	error	строковый литерал не закрыт	3.2
P1207	error	ошибка синтаксиса Python: {problem}	12.2
P1208	error	f-строка содержит вложенные кавычки того же вида; Primula использует синтаксис Python 3.11	0.2, решение 30
P1209	error	'|{form}' требует тела '{{…}}'	3.3, A `InlineShort`
P1210	error	'|{form}' допустимо только в начале строки	3.3, B.1
P1211	error	сразу после имени команды '{name}' стоит буква	3.2, A `InlineCommand`
P1212	error	после '|{form}' должны быть пробел и текст	A `Heading`, `Clause`, `ListItem`
P1213	error	уровень заголовка {level} слишком глубокий; допустимы уровни 1–6	5.2
P1214	error	'|!' требует аргументов с видом примечания, например |!("warning")	9.1, A `Admonition`
P1215	error	'{{' не закрыта	3.2, A `InlineBody`
P1216	error	'{name}' не закрыто; ожидается '{close}'	3.5
P1217	error	'{found}' не закрывает '{name}'; ожидается '{expected}'	3.5
P1218	error	'{found}': нет открытого окружения, которое можно закрыть	3.5
P1219	error	'|end' не может закрыть '|{kind}'; используйте '|end{kind}'	3.5, решение 28
P1220	error	'|{form}' без соответствующего '|if'	3.6, решение 21
P1221	error	маркер закрытия должен занимать всю строку	3.5, решение 34
P1222	error	строка имеет меньший или иной отступ, чем строка, открывающая блок	3.7, решение 24
P1223	error	блок verbatim не закрыт: нет строки '{marker}'	3.7, решение 34
P1224	error	блок кода не закрыт: нет строки '|end'	2.2, 3.7
P1225	error	'|verbatim' принимает только аргумент end="…" с непустой однострочной строкой	3.7, A `VerbatimBlock`
P1226	error	'{what}' не допускается непосредственно в '|{container}'	6.2, A 3.5
P1227	error	'|&' допустимо только в строке таблицы	6.4
P1228	error	формула '|=(…)' допустима только в ячейке таблицы	6.8
P1229	error	недопустимый заголовок макроса; ожидается '|def имя(параметры):'	A `DefBlock`
P1230	error	заголовок не допускается в окружении пункта	5.3
P1231	error	'{form}' не допускается в окружении списка	5.4, решение 33
P1232	error	уровень списка {level} идёт после уровня {previous}: уровень пропущен	5.4, решение 32

Код	Вид	Сообщение	§
P1233	error	маркер списка '{marker}' не соответствует родительским пунктам; ожидается '{expected}'	5.4, решение 32
P1234	error	на уровне {level} одного списка смешаны маркированные и нумерованные пункты	5.4, решение 32
P1235	error	'{name}' — зарезервированное имя, его нельзя определять	B.3
P1236	warning	эта строка начинает новый абзац после строковой формы	3.4, 14.2
P1238	warning	окружение длиной {lines} строк закрыто анонимным '|end'; напишите '{close}'	3.5, 14.2, решение 35
P1239	error	после '|list(…)' нет списка	5.4
P1240	error	'|{form}' после '|else'	3.6, A `IfBlock`
P1241	error	окружение списка должно начинаться с пункта списка	5.4, решение 33
P1242	error	после '{after}' ожидается {expected}	A
P1243	error	строка таблицы '|:' допустима только в '|table'	6.4, A 3.5
P1244	error	'level=' нельзя сочетать с {count} '<': уровень задаётся либо числом '<', либо level= с одним '<'	3.3, 5.2, решения 105, 114
P2101	error	'{construct}' не допускается в коде Primula	12.2
P2102	error	имя '{name}' не допускается: имена с подчёркиваниями в начале ведут к внутренним механизмам Python	12.2, 12.7
P2103	error	'is' и 'is not' допускаются только с None	12.2
P2104	error	'{statement}' вне функции или цикла	12.2
P2105	error	'{name} …=' на верхнем уровне повторно определяет глобальное имя; глобальные имена неизменяемы	12.7, решение 40
P2106	error	'@export' допускается только как '@export' или '@export(name="…")' над def на верхнем уровне блока кода	10.4, 12.2, решение 135
P2201	error	имя '{name}' не определено	10.4
P2202	error	локальное имя '{name}' прочитано до присваивания	12.7
P2203	error	'{name}' уже определено	10.4
P2204	error	циклическое определение: {chain}	10.4
P2301	error	операция '{op}' не поддерживается для '{left}' и '{right}'	12.3
P2303	error	деление на ноль	12.4
P2304	error	bool не участвует в арифметике; преобразуйте через int(…)	12.4, решение 41
P2305	error	'{op}' не поддерживается между '{left}' и '{right}'	12.3, решение 43
P2307	error	невозможно преобразовать {value} в {type}	12.4, 12.10
P2308	error	недопустимая числовая операция ({detail})	12.4
P2309	error	переполнение числа: в результате больше {limit} значащих цифр	12.4, решение 108
P2310	error	часовые пояса не поддерживаются: {value}	12.3, решение 109
P2311	error	индекс {index} вне диапазона (размер {size})	12.10
P2312	error	ключ {key} не найден	12.10
P2313	error	'{type}' не имеет атрибута '{name}'	12.10, решение 43
P2314	error	у записи нет поля '{name}'	12.6
P2315	error	'{type}' нельзя перебирать	12.10
P2316	error	'{type}' не поддерживает '{operation}'	12.3
P2317	error	'{type}' не может быть ключом словаря	12.3, решение 43
P2318	error	замороженное значение {type} нельзя изменять	12.7, решения 40, 46
P2319	error	{type} нельзя изменять во время перебора	12.7

Код	Вид	Сообщение	§
P2320	error	невозможно распаковать: ожидается значений {expected}, получено {got}	12.2
P2321	error	{function}(): {problem}	12.2
P2322	error	{function}: {problem}	12.10
P2324	error	невозможно присвоить этой цели	12.2
P2325	error	поле '{name}' задано дважды (имена сравниваются после нормализации NFKC)	3.9, 12.6, решение 102
P2330	error	доступ к атрибуту в поле формата не допускается: '{{{field}}}'; используйте индекс, например {{0[name]}}	12.10, решение 45
P2331	error	недопустимый формат: {problem}	12.10
P2340	error	fail: {message}	12.9
P2402	error	превышен лимит рекурсии ({limit})	12.8
P2403	error	коллекция из {size} элементов превышает лимит размера ({limit})	12.8
P2404	error	строка из {size} символов превышает лимит размера ({limit})	12.8
P2406	error	превышен лимит времени одного вычисления ({limit} с)	12.8
P2501	warning	'{name}' присвоено в операторе верхнего уровня, но там не используется; глобальным именем оно не становится	12.7, решение 40
P2502	warning	имя '{name}' смешивает алфавиты ({scripts})	3.9, 14.2, решение 102
P2503	warning	имя '{name}' отличается от '{other}' только похожими буквами	3.9, 14.2, решение 102
P2601	error	неизвестная команда '|{name}': это не встроенная команда и не определённый макрос или функция	3.2
P2602	error	'{name}' — значение типа {type}, а не команда	3.2
P2603	error	значение типа {type} нельзя вставить в текст	4.2, решение 47
P2604	warning	значение типа {type} вставлено в текст без оформления	4.2, 14.2, решение 47
P2605	error	содержимое '{name}' имеет несколько блоков и не может быть вставлено внутри строки	3.2, решение 48
P2606	error	у макроса '{name}' нет параметра 'body', а вызов передаёт тело	3.2, решение 48
P2608	error	'{what}' в ветви |if верхнего уровня: определение не может быть условным	10.4, решение 49
P2609	error	|{command}: {problem}	10.4
P2610	error	экспортировать имена может только фрагмент, подключённый вне |if, |for, |def	10.4, решение 49
P2611	error	when= должно быть логическим значением, а найдено {type}	9.2, решение 29
P2612	error	значение должно быть известно при построении дерева: '{name}' не является параметром заголовка	10.2, решение 50
P2613	error	недопустимый путь '{path}': URL, абсолютные пути и пути вне корня проекта не допускаются	10.1
P2614	error	не удаётся прочитать '{path}': {reason}	10.1
P2615	error	цикл подключений: {chain}	10.1; 11.7 — цикл основ `extends` модулей стилей (решение 133)
P2616	error	'{path}' — файл вида '{kind}', ожидается '{expected}'	10.2, 10.3; 11.7 — основа `extends` не модуль стилей
P2617	error	неизвестный параметр '{name}' фрагмента '{path}'	10.2
P2618	error	часть '{part}' не найдена в '{path}'	10.2

Код	Вид	Сообщение	§
P2619	error	неизвестный ключ '{key}': его нет в карте ключей проекта	9.3
P2620	error	'{name}' экспортируют два фрагмента; разведите их через ns= или rename=	10.4
P2621	error	'{name}' определено в документе и экспортировано фрагментом; используйте override=True или rename=	10.4
P2622	error	имя модуля '{name}' не является идентификатором или зарезервировано	10.3, решение 52
P2623	error	в модуле допускаются только определения: |def, |let, |code:, |import	10.3, решение 52
P2624	error	'{name}' не является публичным именем модуля '{module}'	10.3, решение 52
P2625	error	модуль стандартной библиотеки '{name}' на этом этапе прототипа недоступен	13.1
P2626	error	параметр 'features' должен быть списком строк	9.2, решение 53
P2629	error	'{value}' в столбце '{column}' не является корректным значением типа {type}	10.5, решение 51
P2630	error	некорректные данные: {problem}	10.5, решения 25, 51
P2632	error	|call требует функцию первым аргументом	3.2
P2633	error	вычисленный стиль должен быть именем стиля (строкой), а найдено {type}	4.3
P2634	error	имя параметра '{name}' не является идентификатором	2.3
P2635	warning	в документе остался заполнитель: '{text}'	4.2
P2636	error	заголовок получает уровень {level} после level_shift={shift}; самый глубокий уровень — 6	10.2, 10.7, ПЗ
P2637	error	'{name}' не определено во фрагменте, его нельзя экспортировать	10.4, решение 135
P2638	warning	'@export' в модуле ничего не меняет: каждое имя модуля без '_' в начале открыто	10.3, решение 135
P2639	warning	'{path}' лежит вне проекта: символическая ссылка '{link}' ведёт в '{target}'; храните этот каталог вместе с проектом, чтобы документ можно было собрать снова	10.1, решение 136
P2701	warning	стандартная библиотека не поддерживает язык документа '{lang}'; использованы английские формы	библ. 1, решение 115
P2702	error	неизвестный язык '{lang}' в lang=; поддерживаются: {allowed}	библ. 1, решение 115
P2703	error	{function}: число float не принимается в денежных расчётах	библ. 3
P2704	error	{function}: значение {value} вне диапазона ({range})	библ. 4, 5
P2705	error	неизвестная валюта '{code}'	библ. 3
P2706	error	неизвестная единица '{code}'	библ. 6
P2707	error	единицы '{a}' и '{b}' измеряют разные величины	библ. 6
P2708	error	{function}: падеж '{value}' не поддерживается; допустимы: {allowed}	библ. 4, 5
P2709	error	{function}: неизвестный род '{value}'; допустимы: {allowed}	библ. 4
P2710	error	fmt_date: шаблон показывает время суток {placeholder}	библ. 5
P2711	error	{function}: неизвестный заполнитель '{placeholder}' в шаблоне	библ. 5
P2712	error	{function}: неизвестная форма '{format}'; допустимы: {allowed}	библ. 3, 5
P2713	error	календарь: неизвестный ключ '{key}'; допустимы: {allowed}	библ. 5
P2714	error	календарь: неверное значение '{key}' (weekend: mon … sun; даты без времени суток)	библ. 5

Код	Вид	Сообщение	§
P2715	error	календарь: дата {date} одновременно рабочая и выходная	библ. 5, решение 115
P2716	error	plural: для дробного числа нужна форма frac=	библ. 7, решение 115
P2717	error	{function}: шаг должен быть больше нуля	библ. 2
P2718	error	split: веса — непустой список неотрицательных чисел с положительной суммой	библ. 3
P2719	error	у валюты '{code}' нет формы frac для языка {lang}	библ. 1, 3, решение 115
P2720	error	qty: поддерживается только симметричный допуск (число в tol=)	библ. 6
P2721	error	{function}: аргумент '{arg}' должен быть {expected}, а не {type}	библ. 1–7
P2722	error	{function}: для языка {lang} нужно форм: {count}, задано: {given}	библ. 1, 3, 7
P2723	error	{function}: дробная часть содержит более {places} знаков	библ. 4
P2724	error	{function}: у элемента нет поля '{key}'	библ. 8
P2725	error	{function}: значение типа {type} не может быть ключом группы	библ. 8
P2727	error	{function}: аргумент {value} вне области определения функции	библ. 9
P2728	error	{function}: {size} байт не помещаются в QR-код с коррекцией {ecc} до версии {version}	библ. 10, решение 137
P2729	error	{function}: символ '{char}' нельзя записать в кодировке {encoding}	библ. 10, решение 137
P3101	error	неизвестное свойство '{prop}' стиля вида {kind} '{name}'	11.3, решение 57
P3102	error	недопустимое значение '{prop}': ожидается {expected}	11.3, решение 57
P3103	error	стиль '{name}' определён дважды в одном слое	11.7
P3104	error	'{name}' одновременно стиль абзаца и стиль символов	11.2
P3105	error	неизвестный родительский стиль '{parent}' стиля '{name}'	11.2
P3106	error	родительский стиль '{parent}' стиля '{name}' имеет вид {kind}, ожидается {expected}	11.2
P3107	error	цикл наследования стилей: {chain}	11.2
P3108	error	неизвестный стиль '{name}'	4.3
P3109	error	'{name}' — стиль вида {kind}; здесь нужен стиль вида {expected}	4.3
P3110	error	недопустимое имя стиля '{name}': буква, затем буквы, цифры, '_', '-'	11.2
P3111	error	имя стиля '{name}' уже занято стилем с другими свойствами	11.4, решение 56
P3112	error	неизвестный шаг шкалы {scale} '{step}'	11.4
P3113	error	здесь допустимы только параметры before=, after=, indent=, а не '{name}'	11.4
P3114	error	здесь нельзя определять стили (верхний уровень модуля | import)	11.7, решение 59
P3115	error	неизвестная конструкция '{key}' для structure_styles / | set	4.5, 11.5, решение 60
P3118	warning	прямое форматирование |fmt: изменение стиля в Writer не повлияет на этот текст	4.4, 11.8, 14.2
P3120	error	неизвестный профиль структуры '{name}'; ожидается business или report	5.1

Код	Вид	Сообщение	§
P3121	error	стиль списка '{name}': формат '{format}' (кириллические буквы) требует документа на украинском или русском	11.3, решения 115, 133 (только для стилей, которые документ использует)
P3122	error	у пункта списка {number} нет буквы: уровень допускает {count} букв ('{letters}')	11.2, 5.4, П1
P4101	error	недопустимый ID '{id}': латинская буква, далее буквы, цифры, '_', '-', '.'	5.6
P4102	error	повторный ID '{id}'	5.6
P4103	error	фрагмент с ID '{id}' вставлен дважды без ns=	5.6, решение 74
P4104	error	неизвестный аргумент '{name}' команды {command}	5
P4105	error	аргумент '{name}' команды {command} должен быть {expected}	5
P4201	error	пропущен уровень заголовка: уровень {level} после уровня {previous}	5.2
P4202	warning	текст нумерованного заголовка начинается с номера; номер добавляется автоматически	5.2
P4203	warning	подраздел идёт после пунктов раздела {number}: номера пунктов и подразделов могут совпасть	5.3, решение 68
P4204	error	пропущен уровень пункта: пункт уровня {level} без родительского пункта	5.3
P4205	error	неизвестный пункт '{id}' в continues= (пункт должен идти раньше)	5.3
P4206	error	continues='{id}': цель должна быть пунктом того же раздела, области и уровня	5.3
P4207	error	start= и continues= нельзя использовать вместе	5.4
P4208	error	неизвестный список '{name}' в continues= (список должен идти раньше)	5.4
P4209	error	повторное имя списка '{name}'	5.4
P4210	error	повторное или зарезервированное имя области '{name}'	5.5
P4211	error	буквы приложения '{letter}' нет среди букв приложений '{letters}'	5.5, решение 71
P4212	error	уровень заголовка {value} не является целым числом от 1 до 6	5.2, решение 114
P4301	error	неизвестная цель ссылки '{id}'	5.7
P4302	error	цель ссылки '{id}' исключена условием	5.7, решение 75
P4303	warning	цель ссылки '{id}' исключена условием {condition} (оставлена из-за --review)	5.7, решение 75
P4304	error	неподходящий режим ссылки '{mode}' для цели {kind} '{id}'	5.7, решение 76
P4305	error	нет меток ссылок для языка документа '{language}'	5.7, решение 72
P4306	error	нет формы '{case}' метки '{key}' для языка '{language}'	5.7, решение 72
P4307	error	неизвестный режим ссылки '{mode}'; ожидается number, label, title или page	5.7
P4308	error	ссылка на '{id}' другого документа не может показывать номер страницы (режим 'page')	5.7, решение 121
P4309	error	цель '{id}' найдена в нескольких документах из references: {documents}	5.7, решение 121
P4401	error	неизвестный вид перечня '{kind}'; ожидается figure, table, abbr или index	5.9
P4402	error	недопустимая настройка структуры {setting}: '{value}'; ожидается {expected}	5.1–5.10
P5101	error	недопустимое значение '{name}' в col(): '{value}'	6.3

Код	Вид	Сообщение	§
P5102	error	недопустимая ширина колонки '{value}'; ожидается "40mm", "30%", "*", "auto" или "20mm..60mm"	6.3
P5103	error	строка {row}: ячейки занимают позиций: {used}, свободных позиций: {free} (с учётом объединений)	6.5, решение 83
P5104	error	rows={rows} выходит за конец таблицы (осталось строк: {remaining})	6.5
P5105	error	'{text}' не является значением типа колонки {type}	6.8, решение 77
P5106	error	{name}() — функция формул таблицы; она разрешена только в формуле ячейки |=(…)	6.8, решение 81
P5107	error	неизвестная колонка таблицы '{column}'	6.8
P5108	error	строка {row} вне таблицы (строки 1–{rows})	6.8, решение 81
P5109	error	{function}(): значение типа {type} не является числом	6.8, решение 81
P5110	error	{function}() от пустого списка значений	6.8, решение 81
P5111	error	цикл формул ячеек: {chain}	6.8
P5112	warning	метрики шрифта '{font}' недоступны (нет файла шрифта или fontTools); ширины колонок приближённые	6.3
P5113	warning	минимальные ширины колонок {columns} не помещаются; колонки сужены	6.3
P5114	error	формулу нельзя записать как формулу Writer (live=True)	6.8, решение 82
P5115	error	|table(data=…): в строке {row} нет поля '{field}'	6.2, П4
P5301	warning	сегмент {segment}: нет блока потока '{track}'; в ячейке — заполнитель	7.7, решение 134
P5302	error	'|{command}' {where}	7.7, решение 134
P5303	error	'{what}' не допускается непосредственно в '|{container}'	7.7, решение 134
P5304	error	'|{command}' не допускается внутри сегмента |parallel	7.7, решение 134
P5305	error	{command}: {problem}: {value}	7.7, решение 134
P5306	error	повторный ID сегмента '{id}'	7.7, решение 134
P5307	error	|track("{track}"): {problem}	7.7, решение 134
P5308	warning	стандартная библиотека не поддерживает язык '{lang}' потока '{track}'; используются английские формы	7.7, решение 134
P5310	warning	сегмент {segment}, поток '{track}': {found} там, где в основном потоке {expected}	7.7, решение 134
P5311	warning	сегмент {segment}, поток '{track}': числа и идентификаторы отличаются от основного потока: не хватает {missing}; лишние {extra}	7.7, решение 134
P5312	warning	сегмент {segment}, поток '{track}': ID отличаются от основного потока: не хватает {missing}; лишние {extra}	7.7, решение 134
P5313	warning	сегмент {segment}, поток '{track}': цели ссылок отличаются от основного потока: не хватает {missing}; лишние {extra}	7.7, решение 134
P5314	warning	сегмент {segment}, поток '{track}': имена в |(…) отличаются от основного потока: не хватает {missing}; лишние {extra}	7.7, решение 134
P5315	warning	|parallel: поток '{track}' заканчивается со счётчиком «{kind}» {value}, основной поток — {primary}	7.7, решение 134
P5316	warning	сноски нумеруются заново по {restart}: Writer нумерует сноски потоков |parallel вместе с основным потоком	7.7, 5.10, решение 134
P5320	warning	сегмент {segment}, поток '{track}': перевод устарел — основной блок изменён после подтверждения (rev {rev}, сейчас {now})	7.7, решение 134
P5321	warning	сегмент {segment}, поток '{track}': подтверждённый блок изменён после подтверждения (rev {rev}, сейчас {now})	7.7, решение 134

Код	Вид	Сообщение	§
P6101	error	не удаётся использовать изображение '{path}': {reason}	8.1, решение 85
P6102	error	недопустимое значение '{name}' в {command}: '{value}'	7, 8
P6103	error	неизвестный стиль страницы '{name}'	7.2
P6104	error	смена страницы не разрешена внутри {where}	7.3
P6105	error	после смены страницы до конца документа или окружения нет содержимого	7.3
P6106	error	{command} внутри группы, которая не разрывается	7.4
P6107	warning	rotate= пока не выводится: изображение записано без поворота	8.5
P6108	warning	изображение '{path}' имеет ориентацию EXIF {orientation}; она не применяется — поверните сам файл	8.2, решение 85
P6109	error	недопустимый водяной знак '{value}': ожидается текст или img(…)	8.6, решение 84
P6110	error	недопустимый текст SVG в svg=: {reason}	8.2, решение 131
P6111	error	текст SVG в svg= не может содержать объявлений DOCTYPE или ENTITY	8.2, решение 131
P6112	error	текст SVG в svg= ссылается наружу на '{ref}': допустимы только ссылки '#…' и 'data:'	8.2, решение 131
P6113	error	|{command}: изображение задаётся ровно одним способом: путь, значение img(…), key= или svg=	8.2, 8.3, решение 131
P7001	warning	kind '{kind}' противоречит подрасширению '{name}', которое означает '{implied}'; действует ключ 'kind'	2.2, 14.2, решение 100
P7101	error	файл проекта должен содержать 'kind: project' в заголовке, найдено '{kind}'	10.6, решение 117
P7102	error	недопустимое значение '{key}' в файле проекта: ожидается {expected}	10.6, решение 117
P7103	error	build(): {problem}	10.7, решение 117
P7104	error	источник '{path}' цели сборки нельзя использовать: {reason}	10.7, решение 117
P7105	error	неизвестный параметр '{name}' цели сборки '{source}': он не объявлен ни в документе, ни в проекте	10.6, 10.7, решение 117
P7106	error	параметр '{name}' получает значение типа {type}, а объявлен с типом {expected}	10.6, 10.7, решение 117
P7107	error	цель сборки переопределяет ключ '{key}', которого нет в карте ключей проекта	9.3, 10.7, решение 117
P7108	error	результат '{path}' цели сборки лежит вне каталога проекта	10.7, решение 117
P7109	error	результат '{path}' записывают несколько целей сборки	10.7, решение 117
P7110	error	параметр документа '{name}' имеет тип {type}, а одноимённый параметр проекта — тип {expected}	10.6, решение 117
P7111	error	{function}() доступна только в секции кода файла проекта	10.6, 10.7, решение 117
P7112	error	нет карты ключей: key= требует файла проекта с 'keys'	9.3, решение 117
P7113	error	ключ '{key}' указывает на часть файла ('{value}'); '#часть' допустима только в |include	9.3, решение 117
P7114	error	key= нельзя сочетать с путём или part=	9.3, решение 117
P7115	warning	'{name}' экспортируют несколько фрагментов; on_conflict: {mode} оставляет экспорт {mode}	10.4, решение 117
P7116	error	|document допустима только отдельной строкой в тексте корневого документа (в том числе внутри |if и |for), а не в окружении, части или фрагменте	10.7, решения 117, 118
P7117	warning	ключ '{key}' в заголовке части игнорируется: стили и настройки структуры берутся из корневого документа и проекта	10.7, 14.2, решение 117

Код	Вид	Сообщение	§
P7118	error	недопустимая или повторная область '{scope}' в |document	10.7, 5.6, решения 117, 118
P7119	error	документ '{path}' лежит вне каталога проекта '{project}'	10.6, 14.1, решение 117
P7120	error	{option} нельзя использовать при сборке проекта: результаты задают цели	14.1, решение 117
P7121	error	--dump-project требует файла проекта (project.p.prml)	14.1, решение 117
P7122	warning	в проекте нет целей сборки	10.7, решение 117
P7123	error	--param '{text}' следует записать как ИМЯ=ЗНАЧЕНИЕ	14.1, П2
P7124	error	--param: у документа '{path}' нет параметра '{name}' (ни собственного, ни проекта)	14.1, П2
P7125	error	--param нельзя использовать при сборке проекта: параметры целей задаются в 'targets' и build()	14.1, 10.6, П2
P7126	error	references: '{name}' не является документом проекта (ref_documents: {known})	10.6, решения 121, 129
P7127	error	документ '{name}', на который ссылаются, не удалось вычислить (ошибок: {errors}); соберите его отдельно, чтобы их увидеть	10.6, решение 121
P7128	error	у документа '{name}', на который ссылаются, нет ни источника, ни реестра ({registry})	10.6, решение 121
P7129	warning	источник документа '{name}', на который ссылаются, не найден: тексты ссылок — из его реестра {registry}	10.6, решение 121
P7130	error	extends: '{path}' образует круг файлов проекта	10.6, решение 123
P7131	error	extends: проект должен лежать внутри корня базового проекта '{path}'	10.6, решение 123
P7132	error	--dump-structure и --dump-styles требуют документа, а не файла проекта	14.1, решение 124
P8001	warning	'{construct}' пока не выводится (этап {stage}); его место отмечено заполнителем	15
P8002	error	не удаётся записать '{path}': {reason}	14.1
P8003	error	для экспорта PDF нужен LibreOffice: soffice или libreoffice не найден	14.1
P8004	error	экспорт PDF не удался: {reason}	14.1
P8006	warning	этот блок пока нельзя записать внутри пункта списка, он пропущен	15
P8007	warning	|fmt: неизвестное или недопустимое свойство '{prop}'	4.4
P8008	warning	номера страниц в PDF (оглавление, перечни, ссылки на страницы) не обновлены: {reason}	5.9, 14.1
P8101	warning	'{construct}' не имеет соответствия в экспорте и пропускается (разметка страниц: колонтитулы, номера страниц)	14.1, решение 139
P8102	error	для экспорта в этот формат нужен pandoc: pandoc не найден (https://pandoc.org; --to json пишет AST без него)	14.1, решение 139
P8103	error	pandoc завершился с ошибкой: {reason}	14.1, решение 139
P9101	error	неизвестный вид блока '{kind}'; допустимы: {allowed}	9.1, решение 116
P9102	error	блок пуст: нет текста, hazard= или consequence=	9.1, решение 116
P9103	warning	'{kind}' без hazard=: укажите вид и источник опасности	9.1, 14.2, решение 116
P9104	warning	hazard= и consequence= для '{kind}' игнорируются	9.1, решение 116
P9105	warning	функция модуля стилей admonition() в версии языка 1.1 не используется; блоки оформляются стилями	9.1, решение 116
P9106	warning	этот |if определяет имена в ветках и в режиме --review не показывается со всеми вариантами	9.2, решение 116

Код	Вид	Сообщение	§
P9201	error	неизвестное сокращение '{name}'	9.4, решение 116
P9202	error	сокращение '{name}' определено с разными расшифровками	9.4, решение 116
P9203	error	show=False требует расшифровки: |abbr("{name}", "…", show=False)	9.4, решение 116
P9204	warning	abbreviations(first="short") без |list_of("abbr"): расшифровки нигде не выводятся	9.4, решение 116
P9205	warning	|list_of("abbr"): в документе нет сокращений; перечень не выводится	5.9, 9.4, решение 116
P9206	error	abbreviations({key}=…): недопустимое значение '{value}'; допустимы: {allowed}	9.4, решение 116
P9207	error	|abbr требует сокращение первым аргументом (строка)	9.4, решение 116
P9301	error	|index требует непустой термин (строка) и необязательную строку sub=	9.5, решение 116

Приложение Д. Стили комплекта

Сгенерировано `tools/author_docs.py appendices` из справочника транслятора; не править вручную.

std/styles/default

Встроенная основа: стили конструкций, подключается, если модули стилей не названы.

Основа: нет. Шрифт по умолчанию: Liberation Sans, 11pt.

Стиль	Тип	Свойства в модуле
`AbbreviationList`	table	frame: none, rules: none, padding: [0.5mm, 2mm, 0.5mm, 0mm], repeat_header: false, parent: TableGrid
`Admonition`	paragraph	keep_together: true, parent: Body
`AdmonitionCaution`	paragraph	bold: true, color: #9C6500, parent: AdmonitionText
`AdmonitionDanger`	paragraph	bold: true, color: #C00000, parent: AdmonitionText
`AdmonitionNotice`	paragraph	bold: true, color: #1F4E9E, parent: AdmonitionText
`AdmonitionTable`	table	frame: none, rules: none, padding: [1mm, 2mm, 1mm, 2mm], paragraph: AdmonitionText, may_break: false, space_before: 3mm, space_after: 3mm, parent: TableGrid
`AdmonitionTableCaution`	table	parent: AdmonitionTable
`AdmonitionTableDanger`	table	parent: AdmonitionTable
`AdmonitionTableNotice`	table	parent: AdmonitionTable
`AdmonitionTableWarning`	table	parent: AdmonitionTable
`AdmonitionText`	paragraph	align: left, space_after: 1mm, keep_with_next: false, parent: Body
`AdmonitionWarning`	paragraph	bold: true, color: #E36C09, parent: AdmonitionText
`AppendixDesignation`	paragraph	align: right, keep_with_next: true, space_after: 0mm, parent: Body
`AppendixSubtitle`	paragraph	align: right, keep_with_next: true, space_after: 2mm, parent: Body
`AppendixTitle`	paragraph	bold: true, align: center, keep_with_next: true, space_before: 2mm, space_after: 4mm, parent: Body
`Body`	paragraph	align: justify, line_height: 115%, space_after: 2mm, first_line: 0mm, widows: 2, orphans: 2
`BodyCentered`	paragraph	align: center, parent: Body
`BodyCompact`	paragraph	size: 10pt, line_height: 110%, space_after: 1mm, parent: Body
`BodyIndented`	paragraph	first_line: 10mm, parent: Body
`BodyLeft`	paragraph	align: left, parent: Body
`BodyRight`	paragraph	align: right, parent: Body
`BulletList`	list	levels: {1: {type: bullet, bullet: •, left: 5mm, hanging: 5mm}, 2: {type: bullet, bullet: –, left: 10mm, hanging: 5mm}, 3: {type: bullet, bullet: ◦, left: 15mm, hanging: 5mm}}
`Checklist`	list	paragraph: ListText, levels: {1: {type: bullet, bullet: ☐, left: 8mm, hanging: 5mm}}
`Clause`	paragraph	parent: Body
`Code`	character	font: Liberation Mono
`Contents1`	paragraph	align: left, space_after: 1mm, parent: Body
`Contents2`	paragraph	left: 5mm, parent: Contents1
`Contents3`	paragraph	left: 10mm, parent: Contents1
`Contents4`	paragraph	left: 15mm, parent: Contents1

Стиль	Тип	Свойства в модуле
`Contents5`	paragraph	left: 20mm, parent: Contents1
`Contents6`	paragraph	left: 25mm, parent: Contents1
`ContentsTitle`	paragraph	bold: true, size: 12pt, align: center, keep_with_next: true, space_after: 4mm, parent: Body
`DashList`	list	paragraph: ListText, levels: {1: {type: bullet, bullet: –, left: 7mm, hanging: 4mm}, 2: {type: bullet, bullet: –, left: 14mm, hanging: 4mm}}
`DefinedTerm`	character	bold: true
`Emphasis`	character	italic: true
`Endnote`	paragraph	parent: Footnote
`Figure`	paragraph	align: center, keep_with_next: true, space_before: 2mm, space_after: 1mm, parent: Body
`FigureCaption`	paragraph	align: center, space_before: 1mm, space_after: 2mm, parent: Body
`Footer`	paragraph	size: 9pt, align: center, space_after: 0mm, parent: Body
`Footnote`	paragraph	size: 9pt, align: left, line_height: 100%, space_after: 0mm, left: 4mm, first_line: -4mm, parent: Body
`Group`	block	keep_together: true
`Header`	paragraph	size: 9pt, align: center, space_after: 0mm, parent: Body
`Heading`	paragraph	bold: true, align: left, keep_with_next: true, space_before: 4mm, space_after: 2mm, parent: Body
`Heading1`	paragraph	size: 14pt, parent: Heading
`Heading2`	paragraph	size: 13pt, parent: Heading
`Heading3`	paragraph	size: 12pt, parent: Heading
`Heading4`	paragraph	size: 11pt, parent: Heading
`Heading5`	paragraph	size: 11pt, italic: true, parent: Heading
`Heading6`	paragraph	size: 11pt, bold: false, italic: true, parent: Heading
`IndexEntry`	paragraph	align: left, space_after: 1mm, parent: Body
`IndexEntry2`	paragraph	left: 5mm, parent: IndexEntry
`Landscape`	page	orientation: landscape, parent: Portrait
`LatinLetterList`	list	paragraph: ListText, levels: {1: {type: number, format: a, suffix:), left: 9mm, hanging: 6mm}, 2: {type: bullet, bullet: –, left: 18mm, hanging: 4mm}}
`LetterList`	list	paragraph: ListText, levels: {1: {type: number, format: a, suffix:), left: 9mm, hanging: 6mm}, 2: {type: bullet, bullet: –, left: 18mm, hanging: 4mm}}
`ListText`	paragraph	align: left, space_after: 1mm, parent: Body
`MixedList`	list	paragraph: ListText, levels: {1: {type: number, format: 1, suffix: ., left: 9mm, hanging: 6mm}, 2: {type: bullet, bullet: –, left: 18mm, hanging: 4mm}}
`Note`	paragraph	size: 9pt, italic: true, align: left, parent: Body
`NoteAnchor`	character	position: super
`NoteLabel`	character	
`NoteList`	list	paragraph: Note, levels: {1: {type: number, format: 1, suffix: ., left: 5mm, hanging: 5mm}}
`NumberedList`	list	levels: {1: {type: number, format: 1, suffix: ., left: 6mm, hanging: 6mm}, 2: {type: number, format: 1, suffix: ., left: 12mm, hanging: 8mm}, 3: {type: number, format: a, suffix:), left: 18mm, hanging: 6mm}}
`ParallelHeader`	paragraph	bold: true, align: left, keep_with_next: true, parent: Body
`ParallelTable`	table	width: [percent, 100], frame: none, rules: none, padding: 0mm, repeat_header: true, space_before: 3mm, space_after: 3mm
`Placeholder`	character	background: #FFFF00
`Portrait`	page	size: A4, orientation: portrait, margin: [20mm, 15mm, 20mm, 25mm]

Стиль	Тип	Свойства в модуле
`Preformatted`	paragraph	font: Liberation Mono, size: 9pt, align: left, line_height: 100%, space_after: 0mm, widows: 1, orphans: 1, parent: Body
`Quote`	paragraph	italic: true, left: 7mm, right: 5mm, space_before: 2mm, space_after: 3mm, parent: Body
`Reference`	character	
`ReviewFalse`	character	background: #F8D7D7
`ReviewLabel`	paragraph	size: 8pt, italic: true, color: #555555, align: left, space_after: 0mm, keep_with_next: true, parent: Body
`ReviewTrue`	character	background: #D9F2D9
`SmallPrint`	paragraph	align: left, size: 9pt, line_height: 110%, space_after: 1.5mm, color: #333333, parent: Body
`SmallText`	character	size: 9pt
`StrikeText`	character	strike: true
`Strong`	character	bold: true
`SubClause`	paragraph	left: 5mm, parent: Clause
`Subscript`	character	position: sub
`Superscript`	character	position: super
`TableCaption`	paragraph	align: left, keep_with_next: true, space_before: 2mm, space_after: 1mm, parent: Body
`TableCompact`	table	padding: 0.8mm, paragraph: TableTextCompact, header_paragraph: TableHeaderCompact, parent: TableGrid
`TableGrid`	table	width: [percent, 100], frame: 0.5pt solid #000000, rules: all, padding: 1mm, paragraph: TableText, header_bold: true, repeat_header: true
`TableHeader`	paragraph	bold: true, parent: TableText
`TableHeaderCompact`	paragraph	bold: true, parent: TableTextCompact
`TableNoBorders`	table	frame: none, rules: none, padding: [1mm, 2mm, 1mm, 0mm], repeat_header: false, header_background: none, parent: TableGrid
`TableText`	paragraph	align: left, space_after: 0mm, parent: Body
`TableTextCompact`	paragraph	size: 9pt, line_height: 105%, parent: TableText
`Underline`	character	underline: true

std/styles/business

Договоры, акты, счета, протоколы; поля и отступы — по ДСТУ 4163:2020.

Основа: `std/styles/default`. Шрифт по умолчанию: Liberation Serif, 12pt.

Стиль	Тип	Свойства в модуле
`Approval`	paragraph	left: 100mm, space_after: 0mm, keep_with_next: true, keep_together: true, parent: BodyLeft
`ApprovalTitle`	paragraph	bold: true, space_after: 1mm, parent: Approval
`AttachmentLabel`	paragraph	size: 10pt, space_before: 3mm, keep_with_next: true, parent: BodyLeft
`Body` (дополняет)	paragraph	align: justify, line_height: 115%, space_after: 2mm
`ClauseContinuation`	paragraph	left: 7mm, parent: Body
`DatePlace`	paragraph	space_before: 2mm, space_after: 4mm, keep_with_next: true, parent: BodyLeft
`Details`	paragraph	size: 10pt, line_height: 110%, space_after: 1mm, parent: BodyLeft
`DetailsTitle`	paragraph	bold: true, size: 11pt, space_after: 2mm, keep_with_next: true, parent: Details
`DocumentSubtitle`	paragraph	space_after: 2mm, keep_with_next: true, keep_together: true, parent: BodyCentered

Стиль	Тип	Свойства в модуле
`DocumentTitle`	paragraph	size: 14pt, bold: true, space_before: 3mm, space_after: 2mm, keep_with_next: true, keep_together: true, parent: BodyCentered
`DocumentVersion`	paragraph	size: 9pt, color: #444444, space_after: 4mm, keep_with_next: true, parent: BodyCentered
`FigureCaption` (дополняет)	paragraph	space_after: 3mm, size: 10pt
`Footer` (дополняет)	paragraph	color: #333333
`FormLabel`	paragraph	bold: true, space_after: 1mm, keep_with_next: true, parent: BodyLeft
`Header` (дополняет)	paragraph	color: #333333
`Heading` (дополняет)	paragraph	align: left, keep_together: true
`Heading1` (дополняет)	paragraph	size: 12pt, space_before: 4mm, space_after: 2mm
`Heading2` (дополняет)	paragraph	size: 12pt, space_before: 3mm, space_after: 1.5mm
`Heading3` (дополняет)	paragraph	size: 12pt, space_before: 2mm, space_after: 1mm
`Heading4` (дополняет)	paragraph	size: 12pt, italic: true, space_before: 2mm, space_after: 1mm
`Illustration`	frame	width: 110mm
`IllustrationSmall`	frame	width: 70mm
`IllustrationWide`	frame	width: 160mm
`Logo`	frame	width: 40mm
`LogoSmall`	frame	width: 25mm
`Note` (дополняет)	paragraph	size: 10pt, italic: false, space_before: 1mm
`OrganizationName`	paragraph	bold: true, space_after: 2mm, keep_with_next: true, parent: BodyLeft
`Placeholder` (дополняет)	character	background: #FFF2CC
`Portrait` (дополняет)	page	margin: [20mm, 10mm, 20mm, 30mm], footer_gap: 3mm, footer_min_height: 5mm
`Preamble`	paragraph	space_after: 3mm, parent: Body
`SignatureLine`	paragraph	space_after: 3mm, keep_together: true, parent: BodyLeft
`SignatureRole`	paragraph	space_before: 5mm, space_after: 2mm, keep_with_next: true, keep_together: true, parent: BodyLeft
`TableCaption` (дополняет)	paragraph	space_before: 3mm, space_after: 1.5mm, size: 10pt
`TableForm`	table	cols: [30%, 70%], parent: TableGrid
`TableGrid` (дополняет)	table	frame: 0.5pt solid #999999, padding: 1.5mm, border: 0.5pt solid #999999, header_background: #EEEEEE, header_paragraph: TableHeader, keep_rows: true, space_before: 1mm, space_after: 3mm
`TableParties`	table	cols: [50%, 50%], paragraph: Details, header_paragraph: DetailsTitle, header_background: none, padding: [0.5mm, 3mm, 0.5mm, 0mm], may_break: false, parent: TableNoBorders
`TableSchedule`	table	cols: [35%, 45%, 20%], parent: TableGrid
`TableServices`	table	cols: [52%, 12%, 18%, 18%], parent: TableGrid
`TableText` (дополняет)	paragraph	size: 10pt, line_height: 110%, widows: 1, orphans: 1

std/styles/letter

Служебные письма по ДСТУ 4163:2020.

Основа: `std/styles/business`. Шрифт по умолчанию: Liberation Serif, 12pt.

Стиль	Тип	Свойства в модуле
`BodyIndented` (дополняет)	paragraph	space_after: 1mm
`Executor`	paragraph	space_before: 8mm, space_after: 0mm, keep_together: true, parent: SmallPrint

Стиль	Тип	Свойства в модуле
`LetterFirst`	page	next: Portrait, parent: Portrait
`LetterSubject`	paragraph	bold: true, space_before: 6mm, space_after: 4mm, right: 97mm, keep_with_next: true, parent: BodyLeft
`LetterheadDetails`	paragraph	size: 9pt, line_height: 105%, space_after: 0mm, right: 97mm, parent: BodyLeft
`LetterheadName`	paragraph	bold: true, size: 12pt, space_after: 1mm, right: 97mm, parent: BodyLeft
`OutgoingNumber`	paragraph	size: 11pt, space_before: 4mm, space_after: 0mm, right: 97mm, parent: BodyLeft
`Recipient`	paragraph	left: 90mm, line_height: 100%, space_after: 1.5mm, keep_with_next: true, parent: BodyLeft
`Salutation`	paragraph	space_after: 3mm, keep_with_next: true, parent: BodyCentered

std/styles/technical

Руководства, инструкции, технические описания.

Основа: `std/styles/default`. Шрифт по умолчанию: Liberation Sans, 10.5pt.

Стиль	Тип	Свойства в модуле
`Body` (дополняет)	paragraph	align: left, line_height: 120%, space_after: 2mm
`Code` (дополняет)	character	background: #F2F2F2
`Command`	character	font: Liberation Mono, bold: true
`DefinitionTerm`	paragraph	bold: true, space_after: 0.5mm, keep_with_next: true, parent: Body
`DefinitionText`	paragraph	left: 8mm, space_after: 2mm, parent: Body
`FileName`	character	font: Liberation Mono
`Heading1` (дополняет)	paragraph	size: 15pt, space_before: 6mm, space_after: 3mm
`Heading2` (дополняет)	paragraph	size: 13pt, space_before: 5mm, space_after: 2mm
`Heading3` (дополняет)	paragraph	size: 11.5pt, space_before: 4mm, space_after: 1.5mm
`Key`	character	font: Liberation Mono, bold: true, background: #E8E8E8
`MenuItem`	character	bold: true
`Portrait` (дополняет)	page	margin: [20mm, 15mm, 20mm, 25mm]
`Preformatted` (дополняет)	paragraph	size: 9pt, space_after: 2mm, left: 3mm, right: 3mm, background: #F2F2F2, space_before: 1mm
`ProgramOutput`	paragraph	color: #333333, parent: Preformatted
`StepList`	list	paragraph: ListText, levels: {1: {type: number, format: 1, suffix: ., left: 8mm, hanging: 8mm}, 2: {type: number, format: a, suffix:), left: 16mm, hanging: 6mm}}
`StepResult`	paragraph	italic: true, left: 8mm, space_after: 2mm, parent: Body
`TableGrid` (дополняет)	table	frame: 0.5pt solid #808080, border: 0.5pt solid #808080, header_background: #E6E6E6, header_paragraph: TableHeader
`TableRevisions`	table	cols: [16mm, 24mm, *, 40mm], parent: TableCompact
`Variable`	character	font: Liberation Mono, italic: true

std/styles/report

Отчёты по ДСТУ 3008:2015.

Основа: `std/styles/default`. Шрифт по умолчанию: Liberation Serif, 14pt.

Стиль	Тип	Свойства в модуле
`AppendixDesignation` (дополняет)	paragraph	align: center, first_line: 0mm, bold: true
`AppendixSubtitle` (дополняет)	paragraph	align: center, first_line: 0mm

Стиль	Тип	Свойства в модуле
`AppendixTitle` (дополняет)	paragraph	bold: false, first_line: 0mm
`Body` (дополняет)	paragraph	align: justify, line_height: 150%, space_after: 0mm, first_line: 12.5mm
`Contents1` (дополняет)	paragraph	space_after: 0mm, first_line: 0mm
`ContentsTitle` (дополняет)	paragraph	parent: StructuralHeading
`Figure` (дополняет)	paragraph	first_line: 0mm
`FigureCaption` (дополняет)	paragraph	space_before: 1mm, space_after: 3mm, first_line: 0mm
`Footer` (дополняет)	paragraph	first_line: 0mm
`Footnote` (дополняет)	paragraph	size: 12pt
`Header` (дополняет)	paragraph	size: 12pt, align: right, first_line: 0mm
`Heading` (дополняет)	paragraph	align: left, space_before: 7mm, space_after: 7mm, first_line: 12.5mm, keep_together: true
`Heading1` (дополняет)	paragraph	size: 14pt, odf: {fo:text-transform: uppercase}
`Heading2` (дополняет)	paragraph	size: 14pt
`Heading3` (дополняет)	paragraph	size: 14pt
`Heading4` (дополняет)	paragraph	size: 14pt, italic: false
`ListText` (дополняет)	paragraph	space_after: 0mm, first_line: 0mm, line_height: 150%
`Note` (дополняет)	paragraph	size: 12pt, italic: false, align: justify, line_height: 100%, first_line: 12.5mm
`ParallelHeader` (дополняет)	paragraph	first_line: 0mm
`Portrait` (дополняет)	page	margin: [20mm, 10mm, 20mm, 25mm]
`Preformatted` (дополняет)	paragraph	size: 12pt, first_line: 0mm
`ReportTitle`	page	next: Portrait, parent: Portrait
`StructuralHeading`	paragraph	align: center, first_line: 0mm, page_break_before: true, parent: Heading1
`TableCaption` (дополняет)	paragraph	space_before: 3mm, space_after: 1mm, first_line: 12.5mm
`TableText` (дополняет)	paragraph	first_line: 0mm, size: 12pt, line_height: 100%
`TitlePageName`	paragraph	bold: true, space_before: 40mm, space_after: 6mm, odf: {fo:text-transform: uppercase}, parent: TitlePageText
`TitlePageText`	paragraph	first_line: 0mm, space_after: 2mm, parent: BodyCentered

std/styles/compact

Памятки, справки, шпаргалки.

Основа: `std/styles/default`. Шрифт по умолчанию: Liberation Sans, 9pt.

Стиль	Тип	Свойства в модуле
`Body` (дополняет)	paragraph	align: left, line_height: 112%, space_after: 1mm, widows: 1, orphans: 1
`BulletList` (дополняет)	list	levels: {1: {type: bullet, bullet: •, left: 4mm, hanging: 3mm}, 2: {type: bullet, bullet: –, left: 8mm, hanging: 3mm}, 3: {type: bullet, bullet: ◦, left: 12mm, hanging: 3mm}}
`Footer` (дополняет)	paragraph	size: 7.5pt
`Header` (дополняет)	paragraph	size: 7.5pt
`Heading` (дополняет)	paragraph	space_before: 2.5mm, space_after: 1mm
`Heading1` (дополняет)	paragraph	size: 12pt
`Heading2` (дополняет)	paragraph	size: 10.5pt
`Heading3` (дополняет)	paragraph	size: 9.5pt
`Heading4` (дополняет)	paragraph	size: 9pt

Стиль	Тип	Свойства в модуле
`Heading5` (дополняет)	paragraph	size: 9pt
`Heading6` (дополняет)	paragraph	size: 9pt
`ListText` (дополняет)	paragraph	space_after: 0.5mm
`Note` (дополняет)	paragraph	size: 8pt
`Portrait` (дополняет)	page	margin: [12mm, 12mm, 12mm, 12mm]
`Preformatted` (дополняет)	paragraph	size: 8pt, background: #F2F2F2, left: 1mm, right: 1mm
`SmallPrint` (дополняет)	paragraph	size: 7.5pt
`TableGrid` (дополняет)	table	frame: 0.4pt solid #808080, padding: 0.7mm, border: 0.4pt solid #808080, header_background: #E6E6E6, header_paragraph: TableHeader
`TableText` (дополняет)	paragraph	size: 8.5pt, line_height: 105%
`TableTextCompact` (дополняет)	paragraph	size: 8pt

Глоссарий терминов Primula (ru / uk / en)

Единые термины руководства автора, справки Studio, сообщений транслятора и переводов. Переводчик (человек или модель по зафиксированному промпту этапа 15.5) берёт термин отсюда, а не подбирает синоним. Новый термин сначала вносится сюда. Записи `|…` — конструкции языка, они не переводятся.

ru	uk	en	Пояснение
документ	документ	document	файл `*.prml`, собирается в ODT
фрагмент	фрагмент	fragment	файл `*.f.prml`, подключается `|include`
модуль	модуль	module	файл `*.m.prml`, только определения, `|import`
модуль стилей	модуль стилів	styles module	файл `*.s.prml`
проект	проєкт	project	файл `project.p.prml`: параметры, ключи, цели
часть (сводного документа)	частина	part	`|document` или `|part`
заголовок YAML	заголовок YAML	YAML header	блок между строками `---` в начале файла
ключ заголовка	ключ заголовка	header key	`doc:`, `params:`, `styles:` …
метаданные	метадані	metadata	ключ `doc:`; в проекте `doc_defaults:`
параметр	параметр	parameter	`params:`, `--param`
карта ключей	карта ключів	key map	`keys:` проекта для `key=`
цель (сборки)	ціль	target	`targets:` проекта, `build()`
сборка	збирання	build	`primula build`
проверка	перевірка	check	`primula check`
транслятор	транслятор	translator	программа `primula`
разметка	розмітка	markup	текст с командами `|`
команда	команда	command	`|имя(…)`
длинная форма	довга форма	long form	`|имя(…)`, `|имя(…):`
краткая форма	коротка форма	short form	`|>`, `|*{…}` …
строчная форма	рядкова форма	line form	команда до конца строки
окружение	оточення	environment	`|имя(…):` … `|end имя`
маркер закрытия	маркер закриття	closing marker	`|end`, `|endif` …
аргумент	аргумент	argument	значение в `(…)` команды
тело	тіло	body	`{…}` команды или окружение
выражение	вираз	expression	`|(…)`, аргументы команд
блок кода	блок коду	code block	`|code:` … `|end`
вычислительное ядро	обчислювальне ядро	core (evaluator)	подмножество Python
определение	визначення	definition	имя, заданное кодом, `|let`, `|def`
макрос	макрос	macro	`|def` … `|enddef`
действие	дія	action	оператор верхнего уровня кода, кроме определения: `for`, `if`, вызов
запись	запис	record	значение `record`: поля через точку
экспорт	експорт	export	передача имени фрагмента документу: `@export`, `|export`, `export=True`
декоратор	декоратор	decorator	`@export` над `def`

ru	uk	en	Пояснение
стандартная библиотека	стандартна бібліотека	standard library	функции `num`, `money`, `fmt_date` …
библиотека	бібліотека	library	каталог модулей и данных для нескольких проектов (11)
символическая ссылка	символічне посилання	symbolic link	`lib` → `../shared/`…: подключение библиотеки (решение 136)
документ-тест	документ-тест	test document	документ, который проверяет библиотеку через `fail()`
заголовок (раздела)	заголовок	heading	`|<` … `|<<<<<<`
пункт	пункт	clause	`|>`, нумеруется по уровню
элемент списка	елемент списку	list item	`|-`, `|#`
предупреждающий блок	застережний блок	admonition	`|!("warning")`
приложение (к документу)	додаток	appendix	`|appendix`
перекрёстная ссылка	перехресне посилання	cross reference	`|@("id")`
идентификатор	ідентифікатор	ID	`id=` цели ссылки
сноска	виноска	footnote	`|footnote`
оглавление	зміст	table of contents	`|toc`
таблица	таблиця	table	`|table`
строка (таблицы)	рядок	row	`|:`
ячейка	клітинка	cell	`|&`
столбец	стовпець	column	`cols=`
шапка таблицы	шапка таблиці	header rows	`header=`
итоговые строки	підсумкові рядки	footer rows	`footer=`
формула ячейки	формула клітинки	cell formula	`|=(…)`
стиль	стиль	style	имя в модуле стилей
абзацный стиль	стиль абзацу	paragraph style	`|.Имя:`
символьный стиль	символьний стиль	character style	`|.Имя{…}`
стиль страницы	стиль сторінки	page style	`page_style`, `|page`
слой стилей	шар стилів	style layer	проект → `--style` → документ
набор стилей (комплекта)	набір стилів	styles set	`std/styles/business` … (раздел 11.9)
основа модуля стилей	основа модуля стилів	base (of a styles module)	`extends:` модуля стилей
раздел с колонками	розділ з колонками	section	`|section(cols=…)`
параллельные тексты	паралельні тексти	parallel texts	`|parallel`
поток (параллельного текста)	потік	track	`track(…)`, `|track`
основной поток	основний потік	primary track	`primary=`
сегмент	сегмент	segment	`|segment`
сегмент на всю ширину	сегмент на всю ширину	full-width segment	`span="all"`
проверка соответствия	перевірка відповідності	correspondence check	`parity=`
подтверждение перевода	підтвердження перекладу	confirmation of a translation	`primula parallel --stamp`, `rev=`
перевод устарел	переклад застарів	the translation is outdated	P5320
колонтитул	колонтитул	header / footer	`|header`, `|footer`

ru	uk	en	Пояснение
рисунок	рисунок	figure	`|figure`
изображение	зображення	image	`|img`, `img(…)`
водяной знак	водяний знак	watermark	`|watermark`
заполнитель	заповнювач	placeholder	`|?{…}`
буквальный текст	буквальний текст	verbatim text	`|verbatim`
комментарий	коментар	comment	`|%`, `|comment`
сообщение (диагностики)	повідомлення	diagnostic	`error[P1209]`
ошибка / предупреждение	помилка / попередження	error / warning	серьёзность сообщения
консоль вычислителя	консоль обчислювача	evaluator console	`primula console`